Lab Manual

SUBJECT:Laboratory PracticeI

[314448]

TE IT (2019 Course)

Semester v

Institute Vision

"To satisfy the aspirations of youth force, who wants to lead nation towards prosperity through techno-economic developments"

Institute Mission

"To provide, nurture and maintain an environment of high academic excellence, research and entrepreneurship for all aspiring students which will prepare them to face global challenges, maintaining high ethical and moral standard"

Department Vision

“To be a nucleus nurturing learner to cater current & future digital needs”

Department Mission

1. To groom learners for addressing technical challenges by utilizing knowledge and skill sets.
2. To inculcate professional values to develop effective and efficient organization through best practices.

PEOs & PSOs

PEOs:

PEO1: Graduate shall have the ability to exhibit excellence in professional career by demonstrating a positive representation of their brand.

PEO2: Graduate shall have the ability to learn latest trends coping present and future needs.

PEO3: Graduate shall have sense of social responsibility by balancing the emotional quotient and strengthening the personal traits.

PSOs:

PSO1: Apply appropriate technologies and employ suitable methodologies by managing the information technology resources of an individual or organization for betterment.

PSO2: Anticipate the ever changing trends in information technology and assess the likely utility of new technologies.

PSO3: Develop IT systems that would resolve issues related to socio-economic development and build the nation through digitization.

Syllabus

Savitribai Phule Pune University, Pune **Third Year Information Technology (2019 Course)** **314448 : Laboratory Practice-I (Machine Learning)**		
Teaching Scheme:	**Credit Scheme:**	**Examination Scheme:**
Practical (PR) : 4 hrs/week	**02 Credits**	**PR : 25 Marks** **TW: 25 Marks**

Prerequisites:

1. Python programming language

Course Objectives:

1. The objective of this course is to provide students with the fundamental elements of machine learning for classification, regression, clustering.
2. Design and evaluate the performance of a different machine learning models.

Course Outcomes:

On completion of the course, students will be able to–

CO1: Implement different supervised and unsupervised learning algorithms.

CO2: Evaluate performance of machine learning algorithms for real-world applications.

Guidelines for Instructor's Manual

The faculty member should prepare the laboratory manual for all the experiments and it should be made available to students and laboratory instructor/Assistant.

Guidelines for Student's Lab Journal

1. Students should submit term work in the form of a handwritten journal based on a specified listof assignments.
2. Practical Examination will be based on the term work.
3. Students are expected to know the theory involved in the experiment.
4. The practical examination should be conducted if and only if the journal of the candidate is complete in all respects.

Guidelines for Lab /TW Assessment

1. Examiners will assess the term work based on performance of students considering the parameters such as timely conduction of practical assignment, methodology adopted for implementation of practical assignment, timely submission of assignment in the form of handwritten write-up along with results of implemented assignment, attendance etc.
2. Examiners will judge the understanding of the practical performed in the examination by asking some questions related to theory & implementation of experiments he/she has carried out.
3. Appropriate knowledge of usage of software and hardware related to respective laboratories should be as a conscious effort and little contribution towards Green IT and environment awareness, attaching printed papers of the program in a journal may be avoided. There must be hand-written write-ups for every assignment in the journal. The DVD/CD containing student programs should be attached to the journal by every student and the same to be maintained by the department/lab In-charge is highly encouraged. For reference one or two journals may be maintained with program prints at Laboratory.

HOME

Guidelines for Laboratory Conduction

1. All the assignments should be implemented using python programming language
2. Implement any 4 assignments out of 6
3. Assignment number 4 is compulsory
4. The instructor is expected to frame the assignments by understanding the prerequisites, technological aspects, utility and recent trends related to the topic.
5. The instructor may frame multiple sets of assignments and distribute them among batches of students.
6. All the assignments should be conducted on multicore hardware and 64-bit open-sourcesoftware

Guidelines for Practical Examination

1. Both internal and external examiners should jointly set problem statements for practical examination. During practical assessment, the expert evaluator should give the maximum weightage to the satisfactory implementation of the problem statement.
2. The supplementary and relevant questions may be asked at the time of evaluation to judge the student ‘s understanding of the fundamentals, effective and efficient implementation.
3. The evaluation should be done by both external and internal examiners.

List of Laboratory Assignments

Group A

1. **Assignment on Regression technique**

   Download temperature data from below link. https://www.kaggle.com/venky73/temperatures-of-india?select=temperatures.csv

   This data consists of temperatures of INDIA averaging the temperatures of all places month wise. Temperatures values are recorded in CELSIUS

   A. Apply Linear Regression using suitable library function and predict the Month-wise temperature.
   B. Assess the performance of regression models using MSE, MAE and R-Square metrics
   C. Visualize simple regression model.

2. **Assignment on Classification technique**

   Every year many students give the GRE exam to get admission in foreign Universities. The data set contains GRE Scores (out of 340), TOEFL Scores (out of 120), University Rating (out of 5), Statement of Purpose strength (out of 5), Letter of Recommendation strength (out of 5), Undergraduate GPA (out of 10), Research Experience (0=no, 1=yes), Admitted (0=no, 1=yes). Admitted is the target variable.

   Data Set Available on kaggle (The last column of the dataset needs to be changed to 0 or 1)Data Set : https://www.kaggle.com/mohansacharya/graduate-admissions

   The counselor of the firm is supposed check whether the student will get an admission or not based on his/her GRE score and Academic Score. So to help the counselor to take appropriate decisions build a machine learning model classifier using Decision tree to predict whether a student will get admission or not.

   Apply Data pre-processing (Label Encoding, Data Transformation….) techniques if necessary.

   Perform data-preparation (Train-Test Split)

C. Apply Machine Learning Algorithm
D. Evaluate Model.

3. Assignment on Improving Performance of Classifier Models

A SMS unsolicited mail (every now and then known as cell smartphone junk mail) is any junk message brought to a cellular phone as textual content messaging via the Short Message Service (SMS). Use probabilistic approach (Naive Bayes Classifier / Bayesian Network)to implement SMS Spam Filtering system. SMS messages are categorized as SPAM or HAM using features like length of message, word depend, unique keywords etc.

Download Data -Set from : http://archive.ics.uci.edu/ml/datasets/sms+spam+collection

This dataset is composed by just one text file, where each line has the correct class followed by the raw message.

- **A.** Apply Data pre-processing (Label Encoding, Data Transformation….) techniques if necessary
- **B.** Perform data-preparation (Train-Test Split)
- **C.** Apply at least two Machine Learning Algorithms and Evaluate Models
- **D.** Apply Cross-Validation and Evaluate Models and compare performance.
- **E.** Apply Hyper parameter tuning and evaluate models and compare performance.

4. Assignment on Clustering Techniques

Download the following customer dataset from below link:

Data Set: https://www.kaggle.com/shwetabh123/mall-customers

This dataset gives the data of Income and money spent by the customers visiting a Shopping Mall. The data set contains Customer ID, Gender, Age, Annual Income, Spending Score. Therefore, as a mall owner you need to find the group of people who are the profitable customers for the mall owner. Apply at least two clustering algorithms (based on Spending Score) to find the group of customers.

- **A.** Apply Data pre-processing (Label Encoding , Data Transformation….) techniques if necessary.
- **B.** Perform data-preparation(Train-Test Split)
- **C.** Apply Machine Learning Algorithm
- **D.** Evaluate Model.
- **E.** Apply Cross-Validation and Evaluate Model

5. Assignment on Association Rule Learning

Download Market Basket Optimization dataset from below link.

Data Set: https://www.kaggle.com/hemanthkumar05/market-basket-optimization

This dataset comprises the list of transactions of a retail company over the period of one week. It contains a total of 7501 transaction records where each record consists of the list of items sold in one transaction. Using this record of transactions and items in each transaction, find the association rules between items.

There is no header in the dataset and the first row contains the first transaction, so mentioned header = None here while loading dataset.

- **A.** Follow following steps :
- **B.** Data Preprocessing
- **C.** Generate the list of transactions from the dataset
- **D.** Train Apriori algorithm on the dataset
- **E.** Visualize the list of rules

F. Generated rules depend on the values of hyper parameters. By increasing the minimum confidence value and find the rules accordingly

6. **Assignment on Multilayer Neural Network Model**

Download the dataset of National Institute of Diabetes and Digestive and Kidney Diseases from below link :

Data Set: https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv

The dataset is has total 9 attributes where the last attribute is "Class attribute" having values 0 and 1. (1="Positive for Diabetes", 0="Negative")

- **A.** Load the dataset in the program. Define the ANN Model with Keras. Define at least two hidden layers. Specify the ReLU function as activation function for the hidden layer and Sigmoid for the output layer.
- **B.** Compile the model with necessary parameters. Set the number of epochs and batch size and fit the model.
- **C.** Evaluate the performance of the model for different values of epochs and batch sizes.
- **D.** Evaluate model performance using different activation functions Visualize the model using ANN Visualizer.

Reference Books:

1. Ethem Alpaydin, Introduction to Machine Learning, PHI 2nd Edition-2013
2. Peter Flach: Machine Learning: The Art and Science of Algorithms that Make Sense of Data, Cambridge University Press, Edition 2012.
3. Hastie, Tibshirani, Friedman: Introduction to Statistical Machine Learning with Applications in R, Springer, 2nd Edition 2012
4. Tom M. Mitchell , Machine Learning, 1997, McGraw-Hill, First EditionC. M. Bishop: Pattern Recognition and Machine Learning, Springer 1st Edition-2013.
5. Ian H Witten, Eibe Frank, Mark A Hall: Data Mining, Practical Machine Learning Tools and Techniques, Elsevier, 3rd Edition
6. Hastie, Tibshirani, Friedman: Introduction to Statistical Machine Learning with Applications in R, Springer, 2nd Edition 2012.
7. Kevin P Murphy: Machine Learning – A Probabilistic Perspective, MIT Press, August 2012.
8. Shalev-Shwartz S., Ben-David S., Understanding Machine Learning: From Theory to Algorithms, CUP, 2014
9. Jack Zurada: Introduction to Artificial Neural Systems, PWS Publishing Co. Boston, 2002

Virtual Laboratory :

1. http://vlabs.iitb.ac.in/vlabs-dev/labs/machine_learning/labs/index.php

Savitribai Phule Pune University, Pune Third Year Information Technology (2019 Course) 314448 (B) : Laboratory Practice-I (ADBMS)		
Teaching Scheme:	**Credit Scheme**	**Examination Scheme:**
Practical (PR) :4 hrs/week	**02 Credits**	**PR : 25 Marks** **TW : 25 Marks**

HOME

Prerequisites:

1. Database Management System

Course Objectives:

1. To learn and understand Database Modeling, Architectures.
2. To learn and understand Advanced Database Programming Frameworks.
3. To learn NoSQL Databases (Open source) such as MongoDB.
4. To design and develop application using NoSQL Database.
5. To design data warehouse schema for given system.

Course Outcomes:

On completion of the course, students will be able to

CO1: Understand Advanced Database Programming Languages.

CO2: Master the basic concepts of NoSQL Databases.

CO3: Install and configure database systems.

CO4: Populate and query a database using MongoDB commands.

CO5: Design data warehouse schema of any one real-time: CASE STUDYC

CO6: Develop small application with NoSQL Database for back-end.

Guidelines for Instructor's Manual

The faculty member should prepare the laboratory manual for all the experiments and it should be made available to students and laboratory instructor/Assistant.

Guidelines for Student's Lab Journal

1. Student should submit term work in the form of handwritten journal based on specified list of assignments.
2. Practical Examination will be based on all the assignments in the lab manual
3. Candidate is expected to know the theory involved in the experiment.
4. The practical examination should be conducted if and only if the journal of the candidate is complete in all respects.

Guidelines for Lab /TW Assessment

1. Examiners will assess the student based on performance of students considering the parameters such as timely conduction of practical assignment, methodology adopted for implementation of practical assignment, timely submission of assignment in the form of handwritten write-up along with results of implemented assignment, attendance etc.
2. Appropriate knowledge of usage of software and hardware related to respective laboratory should be checked by the concerned faculty member.
3. As a conscious effort and little contribution towards Green IT and environment awareness, attaching printed papers of the program in journal may be avoided. There must be hand-written write-ups for every assignment in the journal. The DVD/CD containing student's programs should be attached to the journal by every student and same to be maintained by department/lab In-charge is highly encouraged. For reference one or two journals may be maintained with program prints at Laboratory.

Guidelines for Laboratory Conduction

1. Group A assignments are compulsory and should be performed by individual student.
2. Group B case study may be performed in group of 3/4.
3. Mini project of Group C can be implemented using any suitable front-end. But back-end must be MongoDB.

Guidelines for Practical Examination

1. Practical Examination will be based on the all topics covered.
2. Examiners will judge the understanding of the practical performed in the examination by asking some questions related to theory & implementation of experiments he/she has carried out.

List of Laboratory Assignments

Group A : MongoDB

1. Create a database with suitable example using MongoDB and implement

- Inserting and saving document (batch insert, insert validation)
- Removing document
- Updating document (document replacement, using modifiers, up inserts, updating multipledocuments, returning updated documents)
- Execute at least 10 queries on any suitable MongoDB database that demonstrates following:
  - Find and find One (specific values)
  - Query criteria (Query conditionals, OR queries, $not, Conditional semantics)
  - Type-specific queries (Null, Regular expression, Querying arrays)
  - $ where queries
  - Cursors (Limit, skip, sort, advanced query options)

2. Implement Map-reduce and aggregation, indexing with suitable example in MongoDB.
Demonstrate the following:

- Aggregation framework
- Create and drop different types of indexes and explain () to show the advantage of the indexes.

3. Case Study: Design conceptual model using Star and Snowflake schema for any one database.

4. Mini Project

Pre-requisite: Build the mini project based on the requirement document and design prepared as a part of Database Management Lab in second year.

1. Form teams of around 3 to 4 people.

2. Develop the application:

Build a suitable GUI by using forms and placing the controls on it for any application. Proper data entry validations are expected.

Add the database connection with front end. Implement the basic CRUD operations.

3. Prepare and submit report to include: Title of the Project, Abstract, List the hardware and software requirements at the backend and at the front end, Source Code , Graphical User Interface, Conclusion.

Reference Books:

1. Silberschatz A., Korth H., Sudarshan S., "Database System Concepts", 6thEdition, McGraw Hill Publishers, ISBN 0-07-120413-X.

2. Kristina Chodorow, MongoDB The definitive guide, O'Reilly Publications, ISBN:978-93-5110-269-4,2nd Edition.

3. Jiawei Han, Micheline Kamber, Jian Pei "Data Mining: concepts and techniques", 2nd Edition, Publisher: Elsevier/Morgan Kaufmann.

4. http://nosql-database.org/.

PART A:
Machine Learning

Assignment No -1

Title: Regression technique

Problem Statement:

This data consists of temperatures of INDIA averaging the temperatures of all place‟s month wise. Temperatures values are recorded in CELSIUS

a) Apply Linear Regression using suitable library function and predict the Month-wise temperature.
b) Assess the performance of regression models using MSE, MAE and R-Square metrics
c) Visualize simple regression model.

Objective: This assignment will help the students to realize how the Linear Regression can be used and predictions using the same can be performed.

S/W Packages and H/W apparatus used:

Linux OS: Ubuntu/Windows, Jupyter notebook.
PC with the configuration as Pentium IV 1.7 GHz. 128M.B RAM, 40 G.B HDD, 15“““Color Monitor, Keyboard, Mouse

Theory:

Definition of Linear Regression

In layman terms, we can define linear regression as **it is used for learning the linear relationship between the target and one or more forecasters**, and it is probably one of the most popular and well inferential algorithms in statistics. Linear regression endeavours to demonstrate the connection between two variables by fitting a linear equation to observed information. One variable is viewed as an explanatory variable, and the other is viewed as a dependent variable.

Normally, linear regression is divided into two types: Multiple linear regression and Simple linear regression.

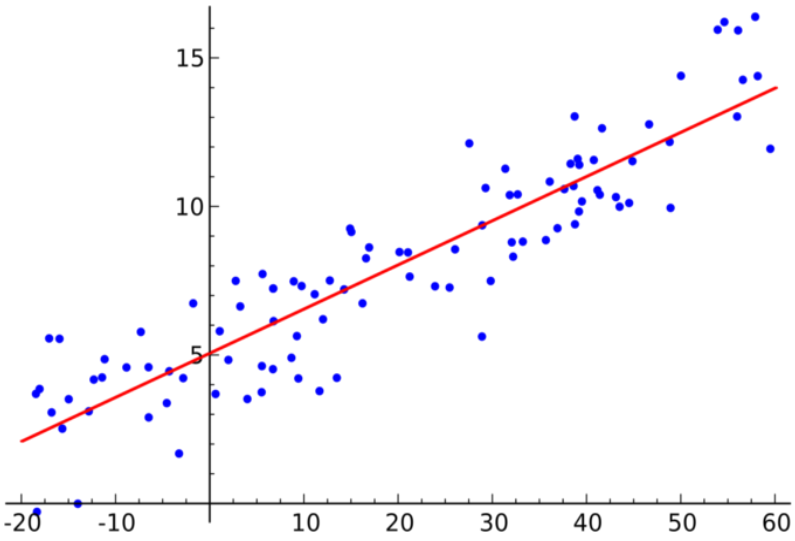
1. **Multiple Linear Regression**

In this type of linear regression, we always attempt to discover the relationship between two or more independent variables or inputs and the corresponding dependent variable or output and the independent variables can be either continuous or categorical.
This linear regression analysis is very helpful in several ways like it helps in foreseeing trends, future values, and moreover predict the impacts of changes.

variable or output. We can discuss this in a simple line as $y = \beta 0 + \beta 1x + \varepsilon$

2. Here, Y speaks to the output or dependent variable, β0 and β1 are two obscure constants that speak to the intercept and coefficient that is slope separately, **Simple Linear Regression**

In simple linear regression, we aim to reveal the relationship between a single independent variable or you can say input, and a corresponding dependent
and the error term is ε Epsilon.We can also discuss this in the form of a graph and here is a sample simple linear regression model graph.



Simple Linear Regression graph

What Actually is Simple Linear Regression?

It can be described as a method of statistical analysis that can be used to study the relationship between two quantitative variables.

Primarily, there are two things which can be found out by using the method of simple linear regression:

1. **Strength of the relationship between the given duo of variables.** (For example, the relationship between global warming and the melting of glaciers)
2. **How much the value of the dependent variable is at a given value of the independent variable.** (For example, the amount of melting of a glacier at a certain level of global warming or temperature)

Regression models are used for the elaborated explanation of the relationship between two given variables. There are certain types of regression models like logistic regression models, nonlinear regression models, and linear regression models. The linear regression model fits a straight line into the summarized data to establish the relationship between two variables.

Assumptions of Linear Regression

To conduct a simple linear regression, one has to make certain assumptions about the data. This is because it is a parametric test. The assumptions used while performing a simple linear regression are as follows:

- **Homogeneity of variance (homoscedasticity)-** One of the main predictions in a simple linear regression method is that the size of the error stays constant. This simply means that in the value of the independent variable, the error size never changes significantly.
- **Independence of observations-** All the relationships between the observations are transparent, which means that nothing is hidden, and only valid sampling methods are used during the collection of data.
- **Normality-** There is a normal rate of flow in the data.

These three are the assumptions of regression methods.

However, there is one additional assumption that has to be taken into consideration while specifically conducting a linear regression.

- **The line is always a straight line-** There is no curve or grouping factor during the conduction of a linear regression. There is a linear relationship between the variables (dependent variable and independent variable). If the data fails the assumptions of

homoscedasticity or normality, a nonparametric test might be used. (For example, the Spearman rank test)

Example of data that fails to meet the assumptions: One may think that cured meat consumption and the incidence of colorectal cancer in the U.S have a linear relationship. But later on, it comes to the knowledge that there is a very high range difference between the collection of data of both the variables. Since the homoscedasticity assumption is being violated here, there can be no linear regression test. However, a Spearman rank test can be performed to know about the relationship between the given variables.

Applications of Simple Linear Regression

1. **Marks scored by students based on number of hours studied (ideally)-** Here marks scored in exams are dependent and the number of hours studied is independent.
2. **Predicting crop yields based on the amount of rainfall-** Yield is a dependent variable while the measure of precipitation is an independent variable.
3. **Predicting the Salary of a person based on years of experience-** Therefore, Experience becomes the independent while Salary turns into the dependent variable.

Limitations of Simple Linear Regression

Indeed, even the best information doesn't recount a total story. Regression investigation is ordinarily utilized in examination to set up that a relationship exists between variables. However, correlation isn't equivalent to causation: a connection between two variables doesn't mean one causes the other to occur. Indeed, even a line in a simple linear regression that fits the information focuses well may not ensure a circumstances and logical results relationship.

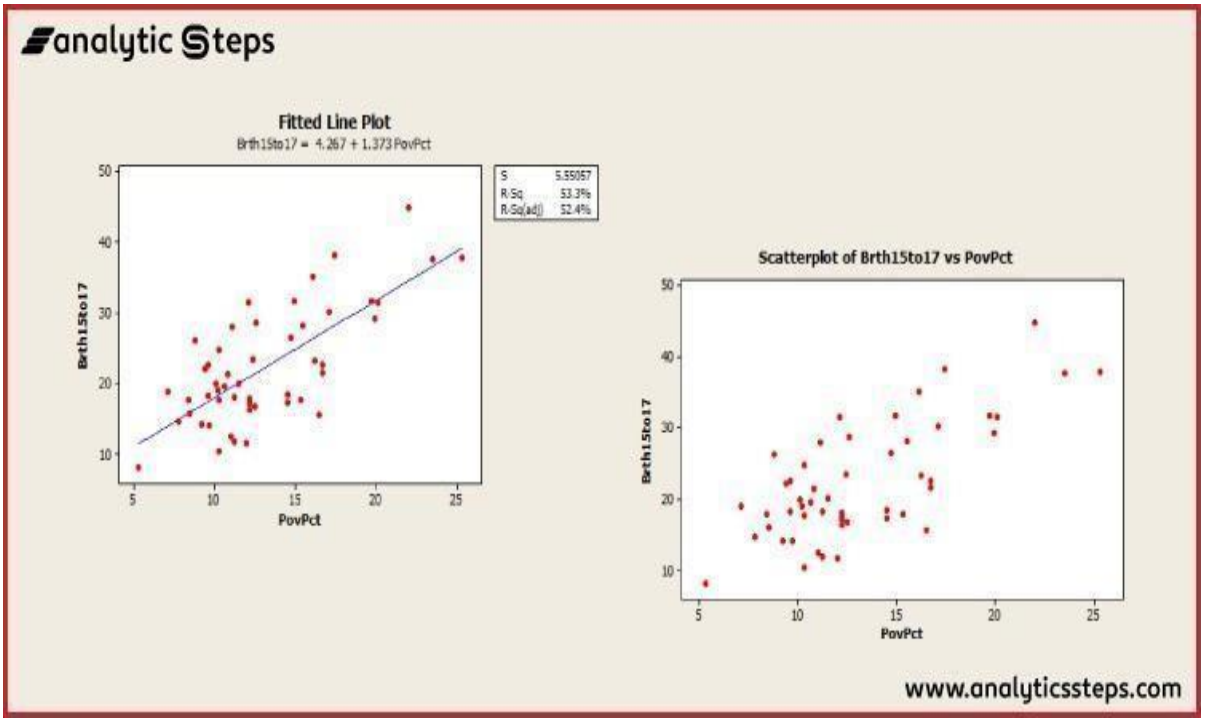
Utilizing a linear regression model will permit you to find whether a connection between variables exists by any means. To see precisely what that relationship is and whether one variable cause another, you will require extra examination and statistical analysis.

Examples of Simple Linear Regression

Now, let‟s move towards understanding simple linear regression with the help of an example. We will take an example of teen birth rate and poverty level data.

This dataset of size n = 51 is for the 50 states and the District of Columbia in the United States. The variables are y = year 2002 birth rate for each 1000 females 15 to 17 years of age and x = destitution rate, which is the percent of the state's populace living in families with wages underneath the governmentally characterized neediness level. (Information source: Mind On Statistics, 3rd version, Utts and Heckard).

Below is the graph (right image) in which you can see the (birth rate on the vertical) is indicating a normally linear relationship, on average, with a positive slope. As the poverty level builds, the birth rate for 15 to 17-year-old females will in general increment too.



Example graph of simple linear regression

Here is another graph (left graph) which is showing a regression line superimposed on the data.

The condition of the fitted regression line is given close to the highest point of the plot. The condition should express that it is for the "average" birth rate (or "anticipated" birth rate would be alright as well) as a regression condition portrays the normal estimation of y as a component of at least one x-variables. In statistical documentation, the condition could be composed y^=4.267+1.373x.

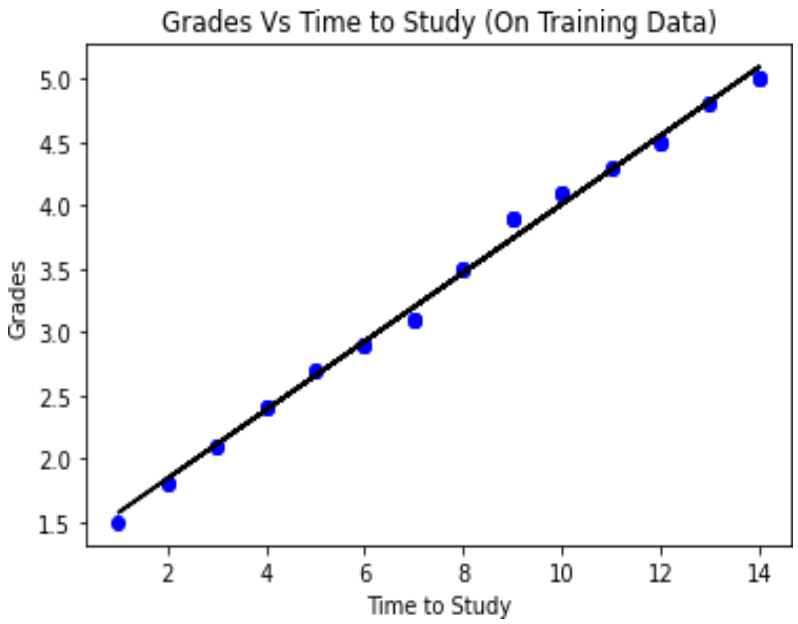
- The translation of the intercept (value=4.267) is that if there were states with a population rate = 0, the anticipated normal for the 15 to 17-year-old birth rate would be 4.267 for those states. Since there are no states with a poverty rate = 0 this understanding of the catch isn't basically significant for this model.
- In the chart with a repression line present, we additionally observe the data that s = 5.55057 and r2 = 53.3%.
- The estimation of s discloses to us generally the standard deviation of the contrasts between the y-estimations of individual perceptions and expectations of y dependent on the regression line. The estimation of r2 can be deciphered to imply that destitution rates "clarify" 53.3% of the noticed variety in the 15 to 17-year-old normal birth paces of the states.

The R2 (adj) value (52.4%) is a change in accordance with R2 dependent on the number of x-variables in the model (just one here) and the example size. With just a single x-variable, the charged R2 isn't significant.

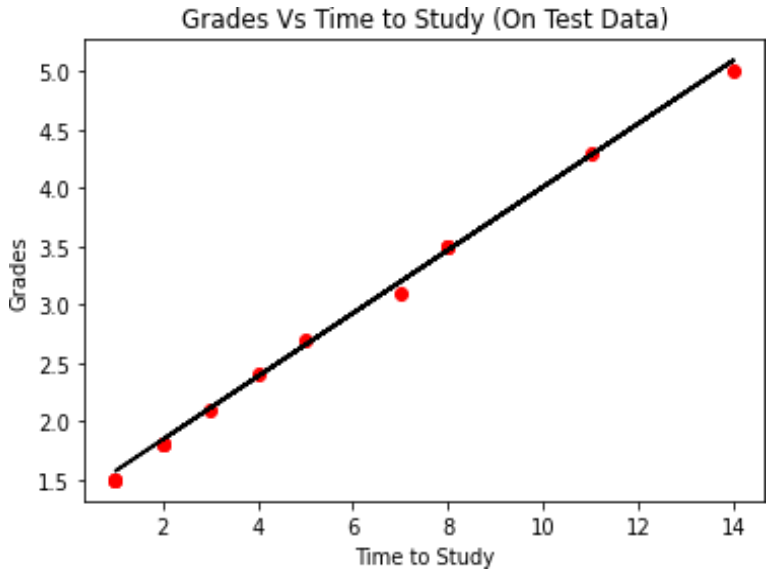
Implementation:

```
#Import Libraries import
numpy as np
import matplotlib.pyplot as plt import pandas
as pd

Importing the csv file

from google.colab import files uploaded =
files.upload()

#Read Student Grades .csv file and divide the data into dependent and inde pendent variables.
data = pd.read_csv('Student_Grades_Data.csv') data.head()

data.shape (50,

2)

X = data.iloc[:, :-1].values y =
data.iloc[:, 1].values
```

```
        2.9, 3.1, 3.5, 3.9, 4.1, 4.3, 4.5, 4.8, 5. , 3.5, 1.8])

#Split the data into training and test datasets from
sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, random_state = 0)


y_test

array([2.1, 3.5, 2.4, 3.5, 3.1, 1.8, 2.7, 5. , 4.3, 1.8, 3.5, 1.8, 1.5,
        1.5, 1.5])

#Fit the Simple Linear Regression Model
from sklearn.linear_model import LinearRegression LinReg =
LinearRegression()
LinReg.fit(X_train, y_train)

#Print the
print(f'a0 = {LinReg.intercept_}') print(f'a1 =
{LinReg.coef_}')

#Predicted grade scores from test dataset y_predict =
LinReg.predict(X_test) y_predict

#Actual grade scores from test dataset y_test

#Grades Vs Time to Study visualization on Training Data plt.scatter(X_train,
y_train, color='Blue') plt.plot(X_train, LinReg.predict(X_train), color='Black')
plt.title('Grades Vs Time to Study (On Training Data)') plt.xlabel('Time to
Study')
plt.ylabel('Grades') plt.show()
```




```
#Grades Vs Time to Study visualization on Test Data plt.scatter(X_test,
y_test, color='Red') plt.plot(X_train, LinReg.predict(X_train),
color='Black') plt.title('Grades Vs Time to Study (On Test Data)')
plt.xlabel('Time to Study')
plt.ylabel('Grades') plt.show()
```



```
#Model Evaluation using R-Square from
sklearn import metrics
r_square = metrics.r2_score(y_test, y_predict) print('R-
Square Error:', r_square)

#Model Evaluation using Mean Square Error (MSE)
print('Mean Squared Error:', metrics.mean_squared_error(y_test, y_predict)
)

#Model Evaluation using Root Mean Square Error (RMSE)
print('Root Mean Squared Error:', np.sqrt(metrics.mean_squared_error(y_tes t, y_predict)))

#Model Evaluation using Mean Absolute Error (MAE)
print('Mean Absolute Error:', metrics.mean_absolute_error(y_test, y_predic t))
```

```
In [1]: from sklearn.model_selection import train_test_split
        from sklearn.linear_model import LinearRegression
        import matplotlib.pyplot as plt
        import numpy as np # linear algebra
        import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
        import seaborn as sns

In [2]: pwd

Out[2]: 'C:\\Users\\admin'

In [3]: file_path=r"C:\Users\admin\Desktop\Pratham\temperatures.csv"
        data=pd.read_csv(file_path)

In [4]: data.describe()

Out[4]:
```

	YEAR	JAN	FEB	MAR	APR	MAY	JUN
count	117.000000	117.000000	117.000000	117.000000	117.000000	117.000000	117.000000
mean	1959.000000	23.687436	25.597863	29.085983	31.975812	33.565299	32.774274
std	33.919021	0.834588	1.150757	1.068451	0.889478	0.724905	0.633132
min	1901.000000	22.000000	22.830000	26.680000	30.010000	31.930000	31.100000
25%	1930.000000	23.100000	24.780000	28.370000	31.460000	33.110000	32.340000
50%	1959.000000	23.680000	25.480000	29.040000	31.950000	33.510000	32.730000
75%	1988.000000	24.180000	26.310000	29.610000	32.420000	34.030000	33.180000
max	2017.000000	26.940000	29.720000	32.620000	35.380000	35.840000	34.480000

```
In [5]: data.head()
```

```
In [5]: data.head()
```

Out[5]:

	YEAR	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC	A
0	1901	22.40	24.14	29.07	31.91	33.41	33.18	31.21	30.39	30.47	29.97	27.31	24.49	
1	1902	24.93	26.58	29.77	31.78	33.73	32.91	30.92	30.73	29.80	29.12	26.31	24.04	
2	1903	23.44	25.03	27.83	31.39	32.91	33.00	31.34	29.98	29.85	29.04	26.08	23.65	
3	1904	22.50	24.73	28.21	32.02	32.64	32.07	30.36	30.09	30.04	29.20	26.36	23.63	
4	1905	22.00	22.83	26.68	30.01	33.32	33.25	31.44	30.68	30.12	30.67	27.52	23.82	

```
In [6]: data.tail()
```

Out[6]:

	YEAR	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC
112	2013	24.56	26.59	30.62	32.66	34.46	32.44	31.07	30.76	31.04	30.27	27.83	25.37
113	2014	23.83	25.97	28.95	32.74	33.77	34.15	31.85	31.32	30.68	30.29	28.05	25.08
114	2015	24.58	26.89	29.07	31.87	34.09	32.48	31.88	31.52	31.55	31.04	28.10	25.67
115	2016	26.94	29.72	32.62	35.38	35.72	34.03	31.64	31.79	31.66	31.98	30.11	28.01

```
In [7]: type(data)
Out[7]: pandas.core.frame.DataFrame

In [8]: data.shape
Out[8]: (117, 18)

In [9]: data.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 117 entries, 0 to 116
Data columns (total 18 columns):
YEAR       117 non-null int64
JAN        117 non-null float64
FEB        117 non-null float64
MAR        117 non-null float64
APR        117 non-null float64
MAY        117 non-null float64
JUN        117 non-null float64
JUL        117 non-null float64
AUG        117 non-null float64
SEP        117 non-null float64
OCT        117 non-null float64
NOV        117 non-null float64
DEC        117 non-null float64
ANNUAL     117 non-null float64
JAN-FEB    117 non-null float64
MAR-MAY    117 non-null float64
JUN-SEP    117 non-null float64
OCT-DEC    117 non-null float64
dtypes: float64(17), int64(1)
memory usage: 16.5 KB

In [10]: count=(data["JAN"]==22).sum()

In [11]: count=(data["JAN"]==22).sum()
         print(count)
1
```

In [12]:
```
column =data
count=column[column==0].count()
print(count)
```

```
YEAR       0
JAN        0
FEB        0
MAR        0
APR        0
MAY        0
JUN        0
JUL        0
AUG        0
SEP        0
OCT        0
NOV        0
DEC        0
ANNUAL     0
JAN-FEB    0
MAR-MAY    0
JUN-SEP    0
OCT-DEC    0
dtype: int64
```

In [13]:
```
data.isnull().sum()
```

Out[13]:
```
YEAR       0
JAN        0
FEB        0
MAR        0
APR        0
MAY        0
JUN        0
JUL        0
AUG        0
SEP        0
OCT        0
NOV        0
DEC        0
ANNUAL     0
JAN-FEB    0
MAR-MAY    0
JUN-SEP    0
OCT-DEC    0
dtype: int64
```

In [14]:
```
data.isnull().head()
```

Out[14]:

	YEAR	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC	AN
0	False	False	False	False	False	False	False	False	False	False	False	False	False	
1	False	False	False	False	False	False	False	False	False	False	False	False	False	
2	False	False	False	False	False	False	False	False	False	False	False	False	False	
3	False	False	False	False	False	False	False	False	False	False	False	False	False	
4	False	False	False	False	False	False	False	False	False	False	False	False	False	

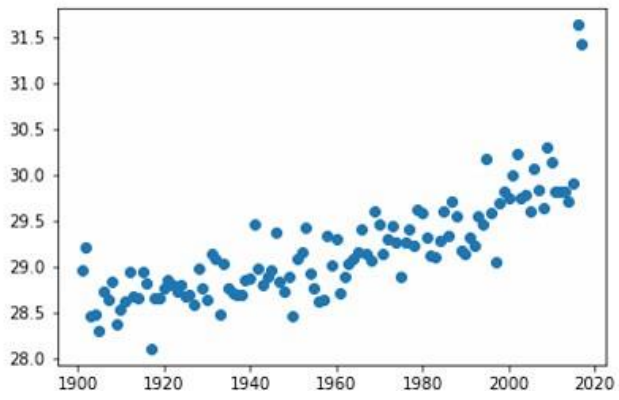
```
In [15]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 117 entries, 0 to 116
Data columns (total 18 columns):
YEAR        117 non-null int64
JAN         117 non-null float64
FEB         117 non-null float64
MAR         117 non-null float64
APR         117 non-null float64
MAY         117 non-null float64
JUN         117 non-null float64
JUL         117 non-null float64
AUG         117 non-null float64
SEP         117 non-null float64
OCT         117 non-null float64
NOV         117 non-null float64
DEC         117 non-null float64
ANNUAL      117 non-null float64
JAN-FEB     117 non-null float64
MAR-MAY     117 non-null float64
JUN-SEP     117 non-null float64
OCT-DEC     117 non-null float64
dtypes: float64(17), int64(1)
memory usage: 16.5 KB
```

```
In [16]: # x=data.iloc[:,1:6]
         # y=data.iloc[:,-1:]
         x=data["YEAR"]
         y=data["ANNUAL"]
```

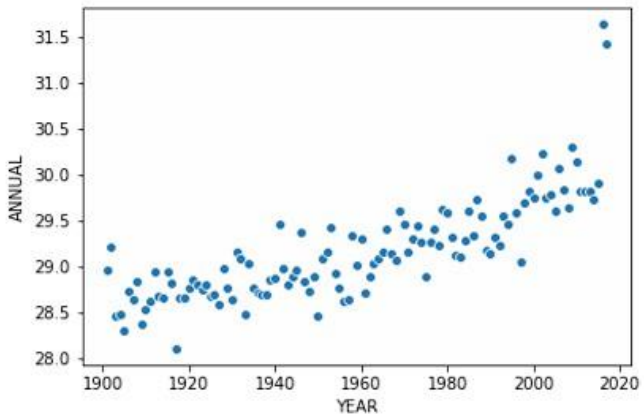
```
In [17]: plt.plot(x,y,'o')
```

```
Out[17]: [<matplotlib.lines.Line2D at 0x2622f628358>]
```



```
In [18]: sns.scatterplot(x=x,y=y)
```

```
Out[18]: <matplotlib.axes._subplots.AxesSubplot at 0x2622f5de860>
```




```
In [19]: type(x)
```

```
Out[19]: pandas.core.series.Series
```

```
In [20]: x.shape
```

```
Out[20]: (117,)
```

```
In [21]: x=x.x.values
```

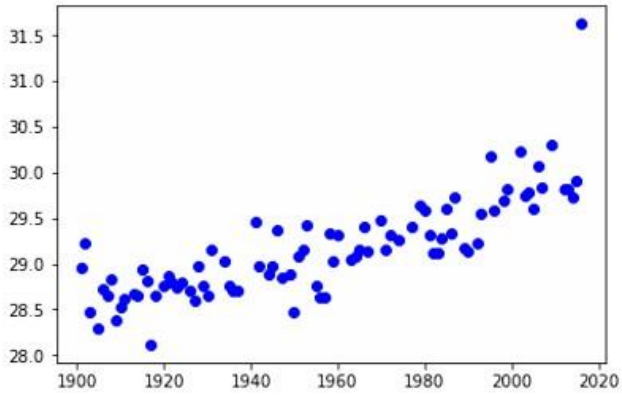
```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
<ipython-input-21-b99b9b95d1cd> in <module>
----> 1 x=x.x.values

C:\ProgramData\Anaconda3\lib\site-packages\pandas\core\generic.py in __getattr__(self, name)
   5065             if self._info_axis._can_hold_identifiers_and_holds_name(name):
   5066                 return self[name]
-> 5067             return object.__getattribute__(self, name)
   5068
   5069     def __setattr__(self, name, value):

AttributeError: 'Series' object has no attribute 'x'
```

```
In [22]: x=x.values

In [23]: x=x.reshape(117,1)

In [24]: x.shape
Out[24]: (117, 1)
```

```
In [25]: type(x)
Out[25]: numpy.ndarray

In [26]: x_train,x_test,y_train,y_test =train_test_split(x,y,test_size=0.25)
         print(f"x Training dataset: {x_train.shape}")
         print(f"y Training dataset: {y_train.shape}")
         print(f"x test dataset: {x_test.shape}")
         print(f"y test dataset: {y_test.shape}")

         x Training dataset: (87, 1)
         y Training dataset: (87,)
         x test dataset: (30, 1)
         y test dataset: (30,)

In [27]: model = LinearRegression()

In [28]: model.fit(x_train,y_train)
Out[28]: LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None,
                          normalize=False)

In [29]: model.coef_ #w
Out[29]: array([0.01271217])
```

```
In [30]: plt.scatter(x_train, y_train, color='blue')
         plt.plot(x_test, y_pred, color='red', linewidth=3)
         plt.title("Temperature vs Year")
         plt.xlabel("Year")
         plt.ylabel("Temperature")
         plt.show()
```

```
---------------------------------------------------------------------------
-
NameError                                 Traceback (most recent call las
t)
<ipython-input-30-2def755fd8e0> in <module>
      1 plt.scatter(x_train, y_train, color='blue')
----> 2 plt.plot(x_test, y_pred, color='red', linewidth=3)
      3 plt.title("Temperature vs Year")
      4 plt.xlabel("Year")
      5 plt.ylabel("Temperature")

NameError: name 'y_pred' is not defined
```

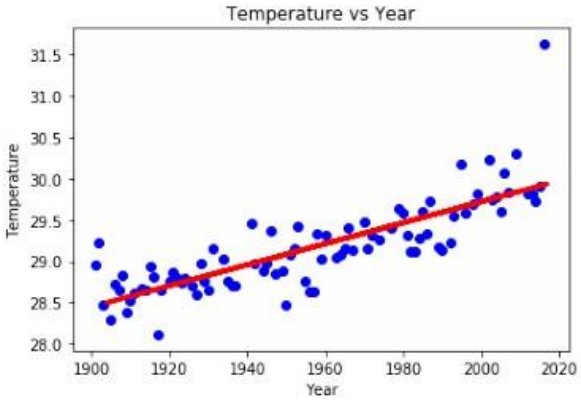



```
In [31]: y_pred = model.predict(x_test)
```

```
In [32]: y_pred.shape
```

```
Out[32]: (30,)
```

In [33]:
```
plt.scatter(x_train, y_train, color='blue')
plt.plot(x_test, y_pred, color='red', linewidth=3)
plt.title("Temperature vs Year")
plt.xlabel("Year")
plt.ylabel("Temperature")
plt.show()
```

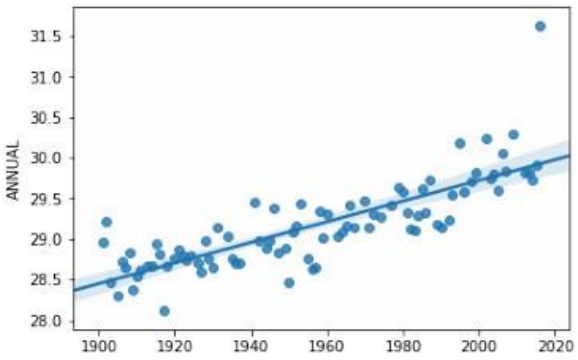


In [34]:
```
sns.regplot(data=df,x=x_train,y=y_train,)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last
t)
<ipython-input-34-9042aa249480> in <module>
----> 1 sns.regplot(data=df,x=x_train,y=y_train,)

NameError: name 'df' is not defined
```

In [35]:
```
```

Out[35]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x2622fd0aac8>
```



```
In [36]: from sklearn.metrics import mean_absolute_error,mean_squared_error,r2_score
         print(f"MSE:  {mean_squared_error(y_test,y_pred)}")
         print(f"MAE:  {mean_absolute_error(y_test,y_pred)}")
         print(f"R-Sqaure :  {r2_score(y_test,y_pred)}")

         MSE:  0.14443462476212654
         MAE:  0.2644742598743015
         R-Sqaure :  0.5979633222180765

In [ ]:
```

Conclusion

Simple linear regression is a regression model that figures out the relationship between one independent variable and one dependent variable using a straight line.

References:

[1]RiyaKumari,https://www.analyticssteps.com/blogs/simple-linear-regression-applications-limitations-examples.

[2]Ethem Alpaydin, Introduction to Machine Learning, PHI 2nd Edition,2013.

[3]Peter Flach: Machine Learning: The Art and Science of Algorithms that Make Sense of Data, Cambridge University Press, Edition 2012.

Assignment No -2

Title: Classification using Machine Learning

Problem Statement:

Perform following operations on given dataset:

a) Apply Data pre-processing (Label Encoding, Data Transformation .....) techniques if necessary.

b) Perform data-preparation (Train-Test Split)

c) Apply Decision tree classification Algorithm

d) Evaluate Model.

Objective:

This assignment will help the students to realize how the decision tree classifier can be used and predictions using the same can be performed.

S/W Packages and H/W apparatus used:

Linux OS: Ubuntu/Windows, Jupyter notebook.

PC with the configuration as Pentium IV 1.7 GHz. 128M.B RAM, 40 G.B HDD, 15""Color Monitor, Keyboard, Mouse

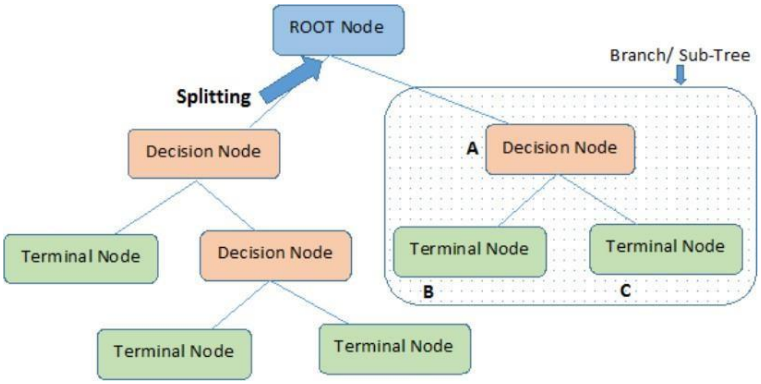
Theory:

Classification:

Classification is **a process of categorizing a given set of data into classes**, It can be performed on both structured or unstructured data. The process starts with predicting the class of given data points. The classes are often referred to as target, label or categories.

What is a Decision Tree?

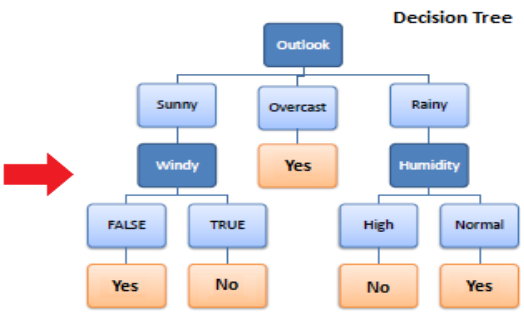
It uses a flowchart like a tree structure to show the predictions that result from a series of feature-based splits. It starts with a root node and ends with a decision made by leaves.[1]



Root Nodes – It is the node present at the beginning of a decision tree. from this node the population starts dividing according to various features.
Decision Nodes – the nodes we get after splitting the root nodes are called Decision Node
Leaf Nodes – the nodes where further splitting is not possible are called leaf nodes or terminal nodes
Sub-tree – just like a small portion of a graph is called sub-graph similarly a sub-section of this decision tree is called sub-tree.
Pruning – It is cutting down some nodes to stop overfitting.

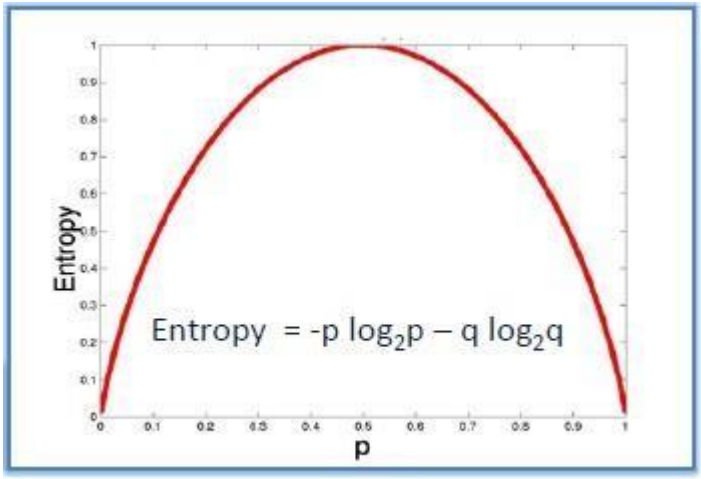
Predictors | Target

Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No



Entropy:

Entropy is used to calculate the homogeneity of a sample. If the sample is completely homogeneous the entropy is zero and if the sample is equally divided it has entropy of one.



Entropy = $-0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$

a) Entropy using the frequency table of one attribute:

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

Play Golf	
Yes	No
9	5

Entropy(PlayGolf) = Entropy (5,9)
= Entropy (0.36, 0.64)
= - (0.36 $\log_2$ 0.36) - (0.64 $\log_2$ 0.64)
= 0.94

b) Entropy using the frequency table of two attributes:

$$E(T,X) = \sum_{c \in X} P(c)E(c)$$

		Play Golf		
		Yes	No	
Outlook	Sunny	3	2	5
	Overcast	4	0	4
	Rainy	2	3	5
				14

E(PlayGolf, Outlook) = **P**(Sunny)*__E__(3,2) + **P**(Overcast)*__E__(4,0) + **P**(Rainy)*__E__(2,3)

= (5/14)*0.971 + (4/14)*0.0 + (5/14)*0.971

= 0.693

Information Gain

The information gain is based on the decrease in entropy after a dataset is split on an attribute. Constructing
a decision tree is all about finding attributes that return the highest information gain (i.e., the most
homogeneous branches).[2]

Step 1: Calculate entropy of the target.

Entropy(PlayGolf) = Entropy (5,9)

= Entropy (0.36, 0.64)

= - (0.36 $\log_2$ 0.36) - (0.64 $\log_2$ 0.64)

= 0.94

Step 2: The dataset is then split on the different attributes. The entropy for each branch is calculated.
Then it is added proportionally, to get total entropy for the split. The resulting entropy is subtracted
from the entropy before the split. The result is the Information Gain, or decrease in entropy.

		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Gain = 0.247			

		Play Golf	
		Yes	No
Temp.	Hot	2	2
	Mild	4	2
	Cool	3	1
Gain = 0.029			

		Play Golf	
		Yes	No
Humidity	High	3	4
	Normal	6	1
Gain = 0.152			

		Play Golf	
		Yes	No
Windy	False	6	2
	True	3	3
Gain = 0.048			

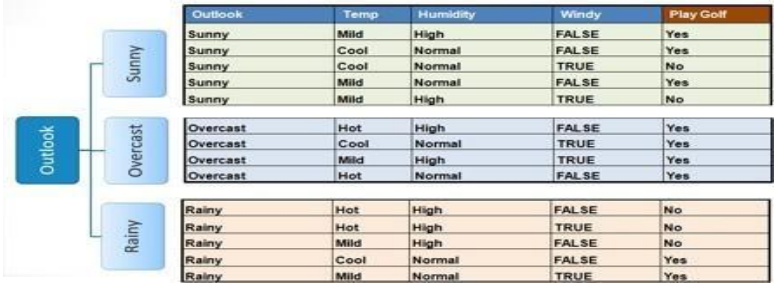
$$Gain(T,X) = Entropy(T) - Entropy(T,X)$$

G(PlayGolf, Outlook) = **E**(PlayGolf) – **E**(PlayGolf, Outlook)

= 0.940 – 0.693 = 0.247

Step 3: Choose attribute with the largest information gain as the decision node, divide the dataset by its
branches and repeat the same process on every branch.

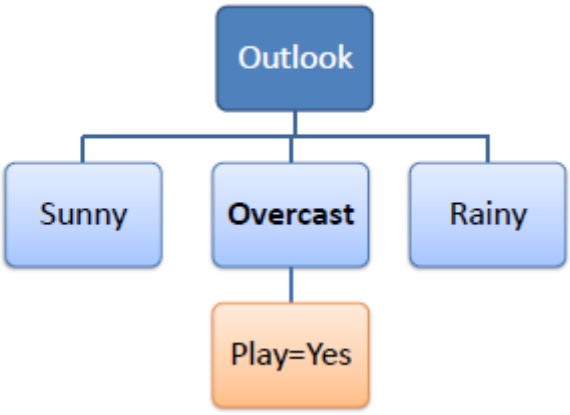
★		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Gain = 0.247			



Outlook	Temp	Humidity	Windy	Play Golf
Sunny	Mild	High	FALSE	Yes
Sunny	Cool	Normal	FALSE	Yes
Sunny	Cool	Normal	TRUE	No
Sunny	Mild	Normal	FALSE	Yes
Sunny	Mild	High	TRUE	No
Overcast	Hot	High	FALSE	Yes
Overcast	Cool	Normal	TRUE	Yes
Overcast	Mild	High	TRUE	Yes
Overcast	Hot	Normal	FALSE	Yes
Rainy	Hot	High	FALSE	No
Rainy	Hot	High	TRUE	No
Rainy	Mild	High	FALSE	No
Rainy	Cool	Normal	FALSE	Yes
Rainy	Mild	Normal	TRUE	Yes

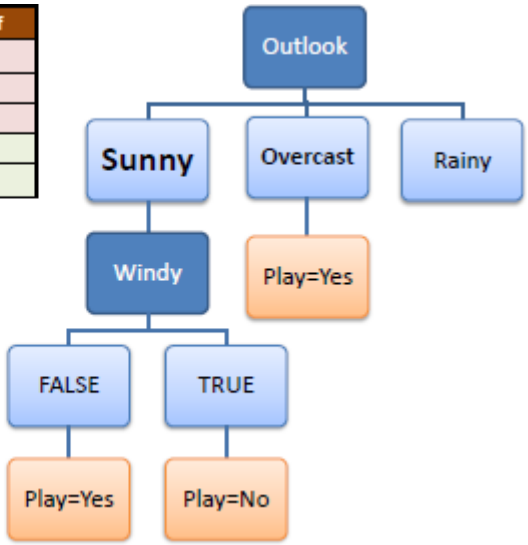
Step 4a: A branch with entropy of 0 is a leaf node.

Temp	Humidity	Windy	Play Golf
Hot	High	FALSE	Yes
Cool	Normal	TRUE	Yes
Mild	High	TRUE	Yes
Hot	Normal	FALSE	Yes



Step 4b: A branch with entropy more than 0 needs further splitting.

Temp	Humidity	Windy	Play Golf
Mild	High	FALSE	Yes
Cool	Normal	FALSE	Yes
Mild	Normal	FALSE	Yes
Cool	Normal	TRUE	No
Mild	High	TRUE	No



Decision Tree to Decision Rules

A decision tree can easily be transformed to a set of rules by mapping from the root node to the leaf nodes one by one.

Step 5: The ID3 algorithm is run recursively on the non-leaf branches, until all data is classified.

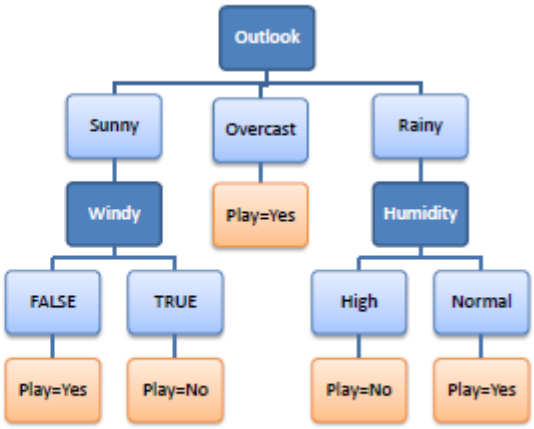
R_1: IF (Outlook=Sunny) AND (Windy=FALSE) THEN Play=Yes

R_2: IF (Outlook=Sunny) AND (Windy=TRUE) THEN Play=No

R_3: IF (Outlook=Overcast) THEN Play=Yes

R_4: IF (Outlook=Rainy) AND (Humidity=High) THEN Play=No

R_5: IF (Outlook=Rain) AND (Humidity=Normal) THEN Play=Yes



Pruning:
It is another method that can help us avoid overfitting. It helps in improving the performance of the tree by cutting the nodes or sub-nodes which are not significant. It removes the branches which have very low importance.
There are mainly 2 ways for pruning:
(i) **Pre-pruning** – we can stop growing the tree earlier, which means we can prune/remove/cut a node if it has low importance **while growing** the tree.
(ii) **Post-pruning** – once our **tree is built to its depth**, we can start pruning the nodes based on their significance.

Implementation:

Importing all the necessary libraries

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import matplotlib.pyplot as plt
```

Importing the csv file

```
df = pd.read_csv('../input/Admission_Predict.csv')
```

 # Check null values in the dataset

```
df.isnull().sum()
```

Out[ ]:
Serial No. 0
GRE Score 0
TOEFL Score 0
LOR 0
CGPA 0
Research 0
Chance of Admit 0
dtype: int64

```
df.columns
```

Index(['Serial No.', 'GRE Score', 'TOEFL Score', 'University Rating', 'SOP',
'LOR ', 'CGPA', 'Research', 'Chance of Admit '],
dtype='object')

Updating chance of admission column
if chance >= 80% CHANCE = 1
if chance < 80% CHANCE = 0

```
dataset.loc[dataset['Chance of Admit '] < 0.8, 'Chance of Admit '] = 0
dataset.loc[dataset['Chance of Admit '] >= 0.8, 'Chance of Admit '] = 1
```

Initializing the variables

```
X = df.drop(['Chance of Admit ','Serial No.'],axis=1)
y = df['Chance of Admit ']
```

Split the data into training and testing set

```
from sklearn.model_selection import train_test_split
X_train,X_test,Y_train,Y_test =
train test split(X,y,test size=0.25,random state=123)
```

```
# importing required libraries
from sklearn.tree import DecisionTreeClassifier
from sklearn import metrics

# Creating Decision Tree classifer object
clf = DecisionTreeClassifier()

# Training Decision Tree Classifer
clf = clf.fit(X_train, Y_train)

#Predicting for the test data
y_pred = clf.predict(X_test)
```

Confusion matrix:

```
print("confusion matrix:\n")
```

```
[[79  3]
 [ 4 39]]
```

```
print("1. Accuracy Score:", metrics.accuracy_score(Y_test, y_pred))

print("2. Precision Score:",metrics.precision_score(Y_test, y_pred))

print("3. Recall Score:", metrics.recall_score(Y_test, y_pred))

print("4. f1 Score:", metrics.f1 score(Y test, y pred))
```

1. Accuracy Score: 0.944
2. Precision Score: 0.9285714285714286
3. Recall Score: 0.9069767441860465
4. f1 Score: 0.9176470588235294

Application:

Helpful in solving classification problems.

References:

[1] Anshul Saini ,Analytics Vidhya,Decision Tree Algorithm – A Complete Guide
[2] Dr. Saed Sayad,https://www.saedsayad.com/decision_tree.htm

```
In [1]: import numpy as np
        import pandas as pd
        import seaborn as sns
        import matplotlib.pyplot as plt
```

```
In [6]: df = pd.read_csv(r"C:\Users\admin\Desktop\TE IT\archive (23)\Admission_Predi
```

```
In [7]: df.head()
```

Out[7]:

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit
0	1	337	118	4	4.5	4.5	9.65	1	0.92
1	2	324	107	4	4.0	4.5	8.87	1	0.76
2	3	316	104	3	3.0	3.5	8.00	1	0.72
3	4	322	110	3	3.5	2.5	8.67	1	0.80
4	5	314	103	2	2.0	3.0	8.21	0	0.65

```
In [8]: df.shape
Out[8]: (500, 9)
```

```
In [9]: df = df.drop('Serial No.',axis=1)
```

```
In [10]: df.shape
Out[10]: (500, 8)
```

```
In [11]: df.head()
```

Out[11]:

	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit
0	337	118	4	4.5	4.5	9.65	1	0.92
1	324	107	4	4.0	4.5	8.87	1	0.76
2	316	104	3	3.0	3.5	8.00	1	0.72
3	322	110	3	3.5	2.5	8.67	1	0.80
4	314	103	2	2.0	3.0	8.21	0	0.65

```
In [12]: df['Chance of Admit '] = [1 if each > 0.75 else 0 for each in df['Chance of
         df.head()
```

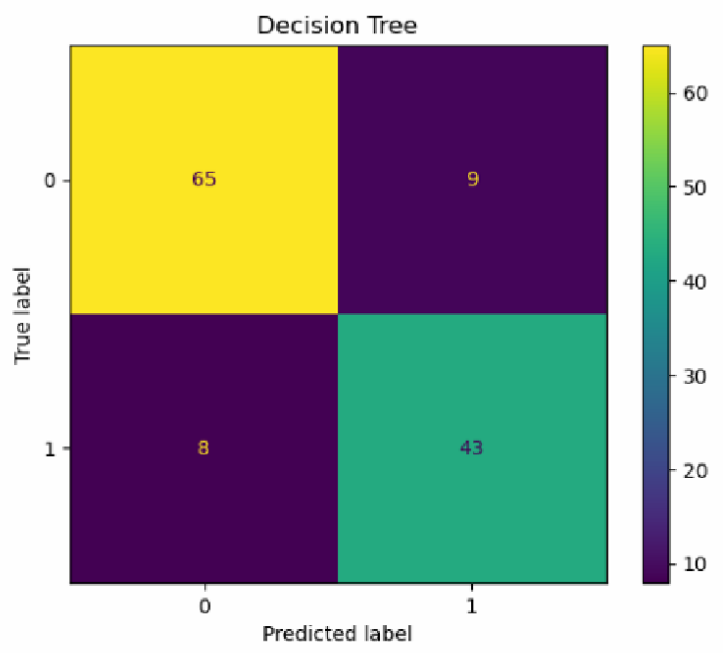
Out[12]:

	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit
0	337	118	4	4.5	4.5	9.65	1	1
1	324	107	4	4.0	4.5	8.87	1	1
2	316	104	3	3.0	3.5	8.00	1	0
3	322	110	3	3.5	2.5	8.67	1	1
4	314	103	2	2.0	3.0	8.21	0	0

```
In [13]: x = df[['GRE Score', 'TOEFL Score', 'University Rating', 'SOP', 'LOR ', 'CGP
               'Research']]

         y = df['Chance of Admit ']

In [14]: from sklearn.model_selection import train_test_split

In [15]: x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=0.25,rando

In [16]: print(f"Size of splitted data")
         print(f"x_train {x_train.shape}")
         print(f"y_train {y_train.shape}")
         print(f"y_train {x_test.shape}")
         print(f"y_test {y_test.shape}")

         Size of splitted data
         x_train (375, 7)
         y_train (375,)
         y_train (125, 7)
         y_test (125,)

In [18]: from sklearn.tree import DecisionTreeRegressor
         from sklearn.ensemble import RandomForestRegressor
         from sklearn.linear_model import LogisticRegression

In [19]: model_dt = DecisionTreeRegressor(random_state=1)
         model_rf = RandomForestRegressor(random_state=1)
         model_lr = LogisticRegression(random_state=1,solver='lbfgs',max_iter=1000)
```

```
In [20]: model_dt.fit(x_train,y_train)
Out[20]: DecisionTreeRegressor(random_state=1)

In [21]: model_rf.fit(x_train,y_train)
Out[21]: RandomForestRegressor(random_state=1)

In [22]: model_lr.fit(x_train,y_train)
Out[22]: LogisticRegression(max_iter=1000, random_state=1)

In [23]: y_pred_dt = model_dt.predict(x_test) #int
         y_pred_rf = model_rf.predict(x_test) #float
         y_pred_lr = model_lr.predict(x_test) #

In [24]: y_pred_rf=[1 if each >0.75 else 0 for each in y_pred_rf]

In [25]: from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score
         from sklearn.metrics import classification_report
```

```
In [27]: ConfusionMatrixDisplay.from_predictions(y_test,y_pred_dt)
         plt.title('Decision Tree')
         plt.show()
         print(f" Accuracy is {accuracy_score(y_test,y_pred_dt)}")
         print(classification_report(y_test,y_pred_dt))
```




```
Accuracy is 0.864
              precision    recall  f1-score   support

           0       0.89      0.88      0.88        74
           1       0.83      0.84      0.83        51

    accuracy                           0.86       125
   macro avg       0.86      0.86      0.86       125
weighted avg       0.86      0.86      0.86       125
```

Assignment No -3

Title: K Means Clustering

Problem Statement:

Perform following operations on given dataset:

a) Apply Data pre-processing (Label Encoding, Data Transformation.....) techniques if necessary.

b) Perform data-preparation (Train-Test Split)

c) Apply Machine Learning Algorithm

d) Evaluate Model.

e) Apply Cross-Validation and Evaluate Model

Objective:

This assignment will help the students to realize how to do Clustering using K Means Clustering algorithm.

S/W Packages and H/W apparatus used:

Linux OS: Ubantu/Windows , Jupyter notebook.

PC with the configuration as Pentium IV 1.7 GHz. 128M.B RAM, 40 G.B HDD, 15""Color Monitor, Keyboard, Mous

Theory:

Introduction to K-means Clustering
K-means clustering is a type of unsupervised learning, which is used when you have unlabeled data (i.e., data without defined categories or groups). The goal of this algorithm is to find groups in the data, with the number of groups represented by the variable K. [1]

The algorithm works iteratively to assign each data point to one of K groups based on the features that are provided. Data points are clustered based on feature similarity.

The results of the K-means clustering algorithm are:
1. The centroids of the K clusters, which can be used to label new data
2. Labels for the training data (each data point is assigned to a single cluster) Rather than defining groups before looking at the data, clustering allows you to find and analyze the groups that have formed organically.

The "Choosing K" section below describes how the number of groups can be determined. Each centroid of a cluster is a collection of feature values which define the resulting groups. Examining the centroid feature weights can be used to qualitatively interpret what kind of group each cluster represents.

This introduction to the K-means clustering algorithm covers:
Common business cases where K-means is used The steps involved in running the algorithm

Some examples of use cases are:

Behavioral segmentation:
- o Segment by purchase history
- o Segment by activities on application, website, or platform.
- o Define personas based on interests
- o Create profiles based on activity monitoring

Inventory categorization:
- o Group inventory by sales activity
- o Group inventory by manufacturing metrics

Sorting sensor measurements:
- o Detect activity types in motion sensors o Group images
- o Separate audio o Identify groups in health monitoring

Detecting bots or anomalies:
- o Separate valid activity groups from bots
- o Group valid activity to clean up outlier detection In addition, monitoring if a tracked data point switches between groups over time can be used to detect meaningful changes in the data.

Algorithm:

The K-means clustering algorithm uses iterative refinement to produce a final result. The algorithm inputs are the number of clusters K and the data set. The data set is a collection of features for each data point. The algorithms start with initial estimates for the K centroids, which can either be randomly generated or randomly selected from the data set. The algorithm then iterates between two steps:

1. **Data assignment step:**

   Each centroid defines one of the clusters. In this step, each data point is assigned to its nearest centroid, based on the squared Euclidean distance. More formally, if ci is the collection of centroids in set C, then each data point x is assigned to a cluster based on

$$\underset{c_i \in C}{\arg\min}\ dist(c_i, x)^2$$

   where dist(·) is the standard (L2) Euclidean distance. Let the set of data point assignments for each i th cluster centroid be Si .

2. **Centroid update step:**

   In this step, the centroids are recomputed. This is done by taking the mean of all data

$$c_i = \frac{1}{|S_i|}\sum_{x_i \in S_i} x_i$$

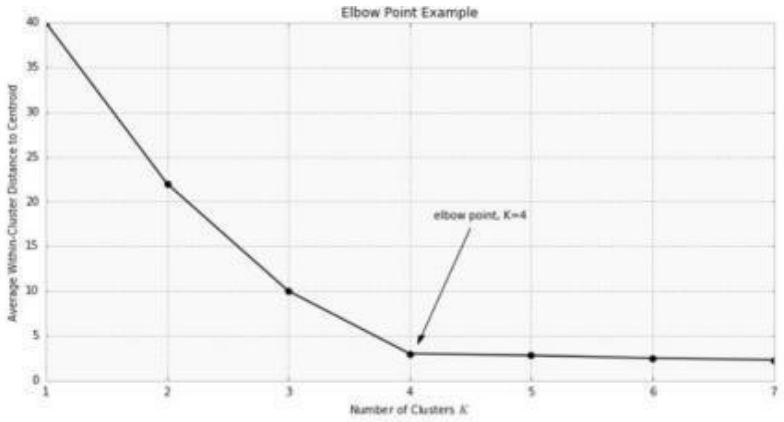
points assigned to that centroid's cluster.The algorithm iterates between steps one and two until a stopping criteria is met (i.e., no data points change clusters, the sum of the distances is minimized, or some maximum number of iterations is reached).

This algorithm is guaranteed to converge to a result. The result may be a local optimum (i.e. not necessarily the best possible outcome), meaning that assessing more than one run of the algorithm with randomized starting centroids may give a better outcome.

Choosing K

The algorithm described above finds the clusters and data set labels for a particular pre-chosen K. To find the number of clusters in the data, the user needs to run the K-means clustering algorithm for a range of K values and compare the results. In general, there is no method for determining exact value of K, but an accurate estimate can be obtained using the following techniques. One of the metrics that is commonly used to compare results across different values of K is the mean distance between data points and their cluster centroid. Since increasing the number of clusters will always reduce the distance to data points, increasing K will always decrease this metric, to the extreme of reaching zero when K is the same as the number of data points. Thus, this metric cannot be used as the sole target. Instead, mean distance to the centroid as a function of K is plotted and the "elbow point," where the rate of decrease sharply shifts, can be used to roughly determine K.

.



Implementation:

Importing Dataset

```
from pandas import read_csv
A=read_csv("E:/DS1/Mall_Customers.csv")
```

Dropping the irrelevant columns

```
B=A.drop(["Customer_ID"],axis=1)
```

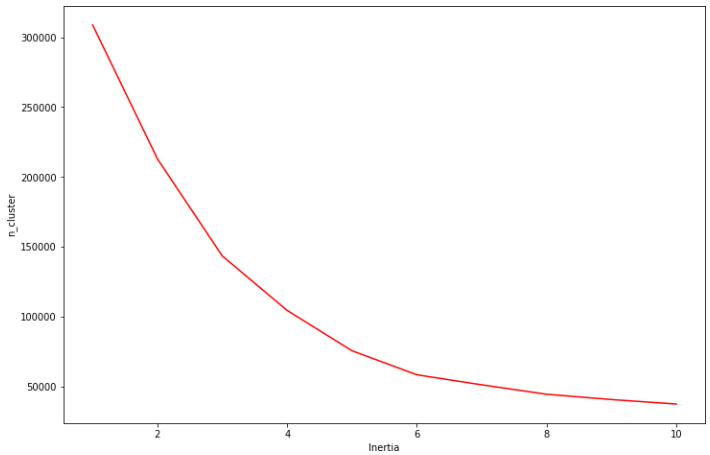
Label encoding

```
# Import label encoder
from sklearn import preprocessing
# label_encoder object knows how to understand word labels.
label_encoder = preprocessing.LabelEncoder()
# Encode labels in column 'species'.
B['Genre']= label_encoder.fit_transform(B['Genre'])

B['Genre'].unique()
array([1, 0], dtype=int64)
```

Finding K

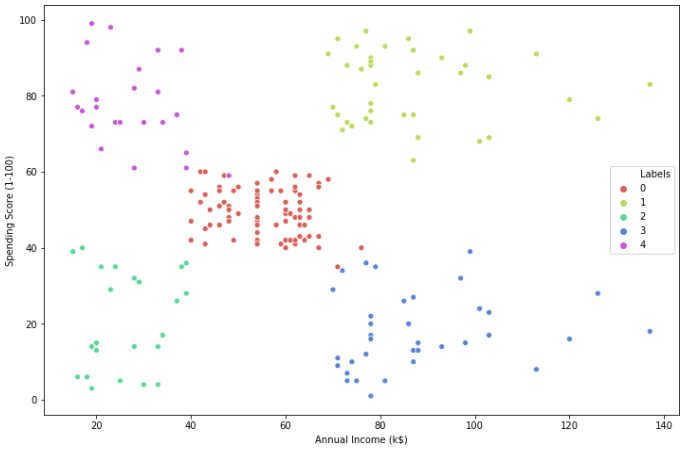
```
from sklearn.cluster import KMeans
cluster = []
for k in range (1, 11):
    kmean = KMeans(n_clusters=k).fit(B)
    cluster.append(kmean.inertia_)

import matplotlib.pyplot as plt plt.figure(
```

```
plt.plot(range(1, 11), cluster, 'r-')
plt.xlabel('Inertia')
plt.ylabel('n_cluster')
plt.show()
```



With above value of K,Create K means clustering model

```
km = KMeans(n_clusters=5).fit(B)
B['Labels'] = km.labels_
import seaborn as sns
plt.figure(figsize=(12, 8))
sns.scatterplot(B['Annual Income (k$)'], B['Spending Score (1-100)'], hue=B['Labels'],
palette=sns.color_palette('hls', 5))
plt.show()
```



In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [3]:
```
df = pd.read_csv(r'C:\Users\admin\Desktop\TE IT\Mall_Customers.csv')
```

In [4]:
```
df
```

Out[4]:

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40
...	...	...	...	...	...
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

200 rows × 5 columns

In [5]:
```
x = df.iloc[:,3:]
x
```

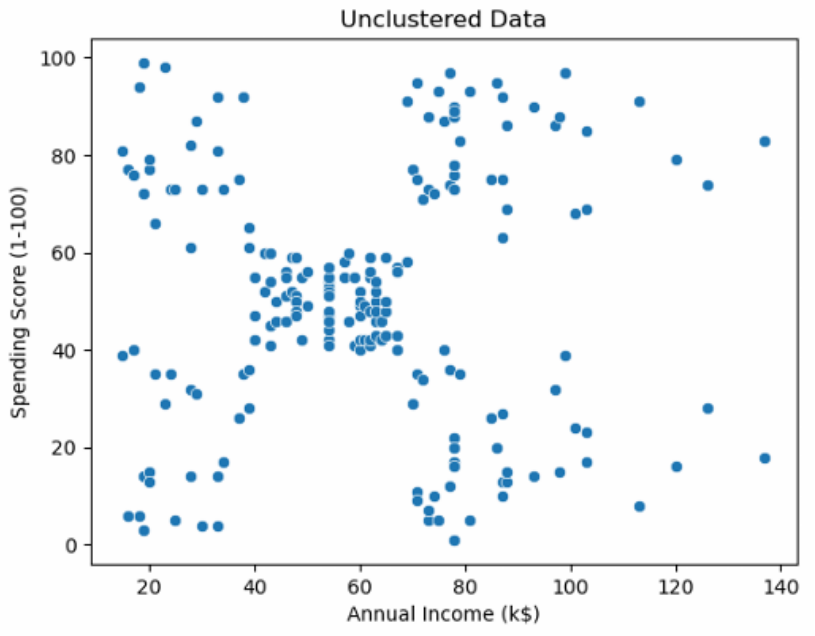
Out[5]:

	Annual Income (k$)	Spending Score (1-100)
0	15	39
1	15	81
2	16	6
3	16	77
4	17	40
...	...	...
195	120	79
196	126	28
197	126	74
198	137	18
199	137	83

200 rows × 2 columns

```
In [6]: plt.title('Unclustered Data')
        sns.scatterplot(x=x['Annual Income (k$)'],y=x['Spending Score (1-100)'])
```

```
Out[6]: <AxesSubplot:title={'center':'Unclustered Data'}, xlabel='Annual Income (k
        $)', ylabel='Spending Score (1-100)'>
```




```
In [7]: from sklearn.cluster import KMeans, AgglomerativeClustering
```

```
In [8]: km = KMeans(n_clusters=4)
```

```
In [9]: km.fit_predict(x)
```

```
Out[9]: array([3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0,
               3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0, 3, 0,
               3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
               0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
               0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
               0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 1, 2, 1, 2, 1, 2, 1, 2,
               1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2,
               1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2,
               1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2,
               1, 2])
```

```
In [10]: km.inertia_
```

```
Out[10]: 73679.78903948837
```

In [11]:
```
sse =[]
for k in range(1,16):
    km = KMeans(n_clusters=k)
    km.fit_predict(x)
    sse.append(km.inertia_)
```

```
C:\Users\admin\anaconda3\lib\site-packages\sklearn\cluster\_kmeans.py:103
6: UserWarning: KMeans is known to have a memory leak on Windows with MKL,
when there are less chunks than available threads. You can avoid it by set
ting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```

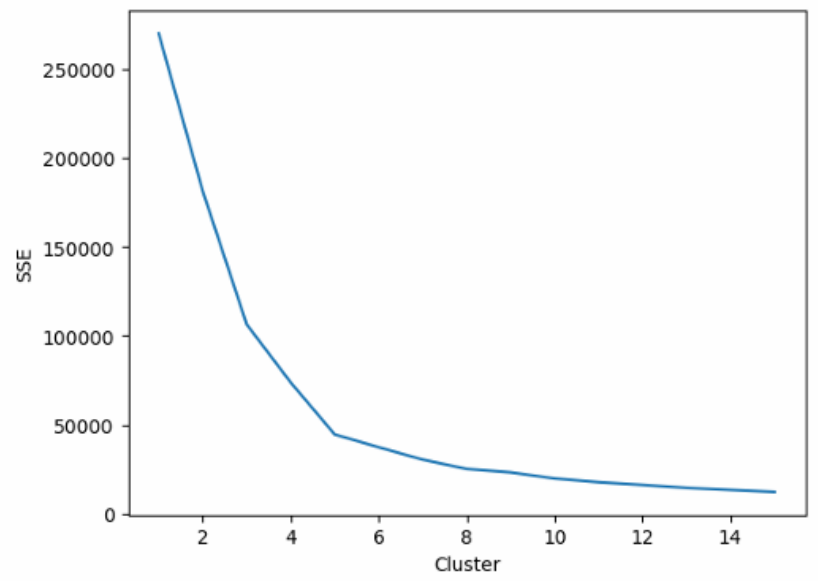
In [12]:
```
sse
```

Out[12]:
```
[269981.28,
 181363.595959596,
 106348.37306211119,
 73679.78903948837,
 44448.45544793371,
 37233.81451071001,
 30273.39431207004,
 25030.383098520328,
 23083.56515395632,
 19641.456216651564,
 17553.829587348137,
 16036.883346746632,
 14367.900783072086,
 13194.164055630361,
 12080.128036094342]
```

In [13]:
```
sns.lineplot(range(1,16),y = sse)
plt.xlabel('Cluster')
plt.ylabel('SSE')
```

```
C:\Users\admin\anaconda3\lib\site-packages\seaborn\_decorators.py:36: Futu
reWarning: Pass the following variable as a keyword arg: x. From version
0.12, the only valid positional argument will be `data`, and passing other
arguments without an explicit keyword will result in an error or misinterp
retation.
  warnings.warn(
```

Out[13]: Text(0, 0.5, 'SSE')



In [14]:
```
from sklearn.metrics import silhouette_score
```

In [15]:
```
silh =[]
for k in range(2,16):
    km = KMeans(n_clusters=k)
    labels = km.fit_predict(x)
    score = silhouette_score(x, labels)
    silh.append(score)
```

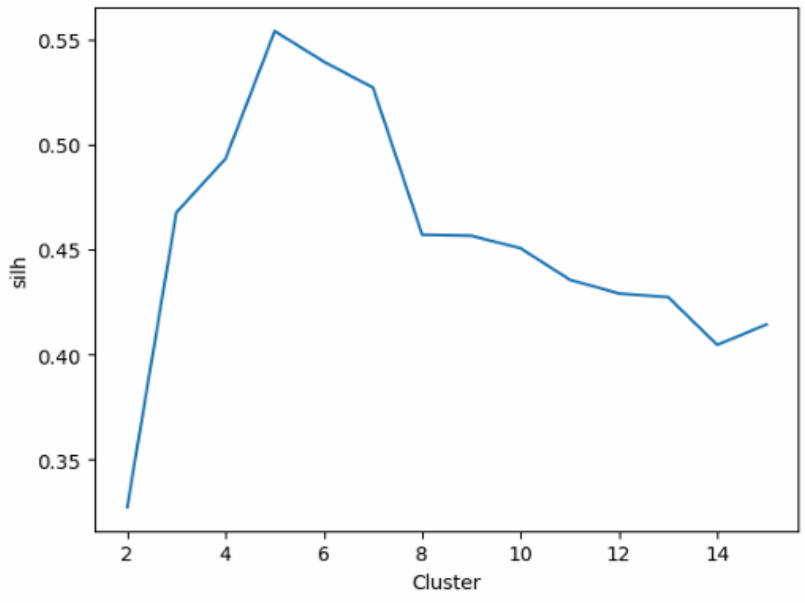
In [16]:
```
silh
```

Out[16]:
```
[0.3273163942500746,
 0.46761358158775435,
 0.4931963109249047,
 0.553931997444648,
 0.5393922132561455,
 0.5270287298101395,
 0.45697007065559897,
 0.4565077334305076,
 0.45056557470336733,
 0.43560008750473395,
 0.42903470410496636,
 0.42735285631755887,
 0.40455775705535274,
 0.4142595565249202]
```

In [17]:
```
sns.lineplot(range(2,16),y = silh)
plt.xlabel('Cluster')
plt.ylabel('silh')
```

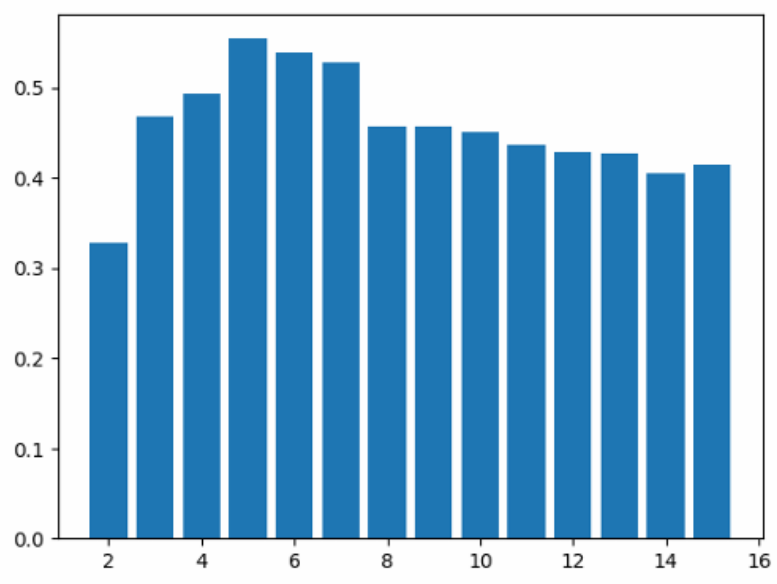
```
C:\Users\admin\anaconda3\lib\site-packages\seaborn\_decorators.py:36: Futu
reWarning: Pass the following variable as a keyword arg: x. From version
0.12, the only valid positional argument will be `data`, and passing other
arguments without an explicit keyword will result in an error or misinterp
retation.
  warnings.warn(
```

Out[17]: Text(0, 0.5, 'silh')



```
In [18]: plt.bar(range(2,16,1),silh)
```

```
Out[18]: <BarContainer object of 14 artists>
```




```
In [19]: km = KMeans(n_clusters=5,random_state=1)
```

```
In [20]: labels = km.fit_predict(x)
```

```
In [21]: km.labels_
```

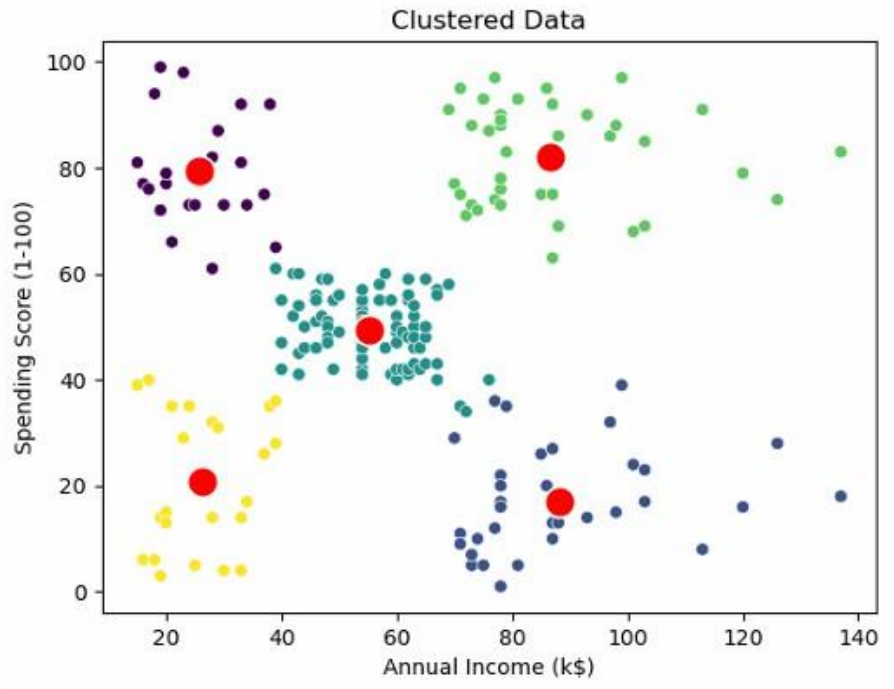
```
Out[21]: array([4, 0, 4, 0, 4, 0, 4, 0, 4, 0, 4, 0, 4, 0, 4, 0, 4, 0, 4, 0,
                4, 0, 4, 0, 4, 0, 4, 0, 4, 0, 4, 0, 4, 0, 4, 0, 4, 0, 4, 2,
                4, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
                2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
                2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
                2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 1, 3, 2, 3, 1, 3, 1, 3,
                2, 3, 1, 3, 1, 3, 1, 3, 1, 3, 2, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3,
                1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3,
                1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3,
                1, 3])
```

```
In [22]: cent = km.cluster_centers_
```

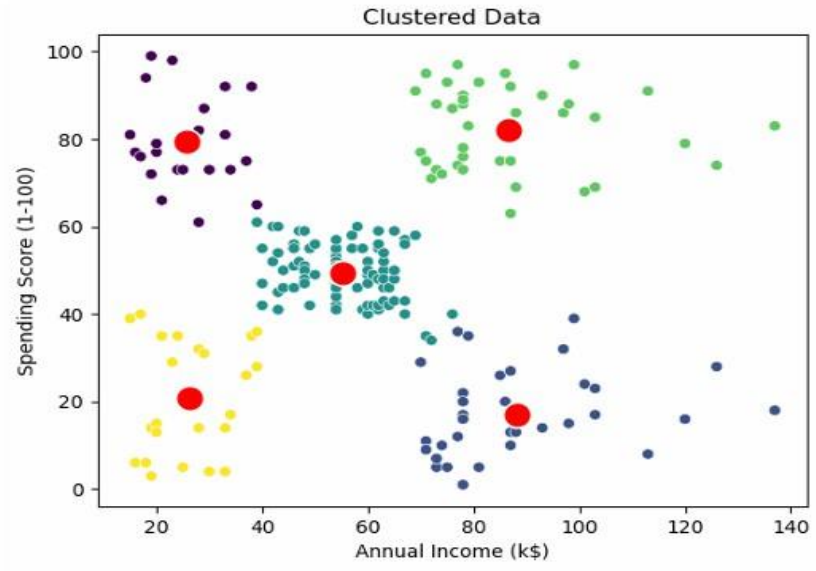
```
In [23]: plt.title('Clustered Data')
         sns.scatterplot(x=x['Annual Income (k$)'],y=x['Spending Score (1-100)'],c=la
         sns.scatterplot(cent[:,0],cent[:,1], s=200, color='red')
```

```
C:\Users\admin\anaconda3\lib\site-packages\seaborn\_decorators.py:36: Futu
reWarning: Pass the following variables as keyword args: x, y. From versio
n 0.12, the only valid positional argument will be `data`, and passing oth
er arguments without an explicit keyword will result in an error or misint
erpretation.
  warnings.warn(
```

```
Out[23]: <AxesSubplot:title={'center':'Clustered Data'}, xlabel='Annual Income (k
         $)', ylabel='Spending Score (1-100)'>
```



```
In [23]: plt.title('Clustered Data')
         sns.scatterplot(x=x['Annual Income (k$)'],y=x['Spending Score (1-100)'],c=la
         sns.scatterplot(cent[:,0],cent[:,1], s=200, color='red')

         C:\Users\admin\anaconda3\lib\site-packages\seaborn\_decorators.py:36: Futu
         reWarning: Pass the following variables as keyword args: x, y. From versio
         n 0.12, the only valid positional argument will be `data`, and passing oth
         er arguments without an explicit keyword will result in an error or misint
         erpretation.
           warnings.warn(

Out[23]: <AxesSubplot:title={'center':'Clustered Data'}, xlabel='Annual Income (k
         $)', ylabel='Spending Score (1-100)'>
```



In [24]:
```
df[labels==0]
```

Out[24]:

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
1	2	Male	21	15	81
3	4	Female	23	16	77
5	6	Female	22	17	76
7	8	Female	23	18	94
9	10	Female	30	19	72
11	12	Female	35	19	99
13	14	Female	24	20	77
15	16	Male	22	20	79
17	18	Male	20	21	66
19	20	Female	35	23	98
21	22	Male	25	24	73
23	24	Male	31	25	73
25	26	Male	29	28	82
27	28	Male	35	28	61
29	30	Female	23	29	87
31	32	Female	21	30	73
33	34	Male	18	33	92
35	36	Female	21	33	81
37	38	Female	30	34	73
39	40	Female	20	37	75
41	42	Male	24	38	92
45	46	Female	24	39	65

In [25]:
```
agl = AgglomerativeClustering(n_clusters=5)
```

In [26]:
```
alabels = agl.fit_predict(x)
```

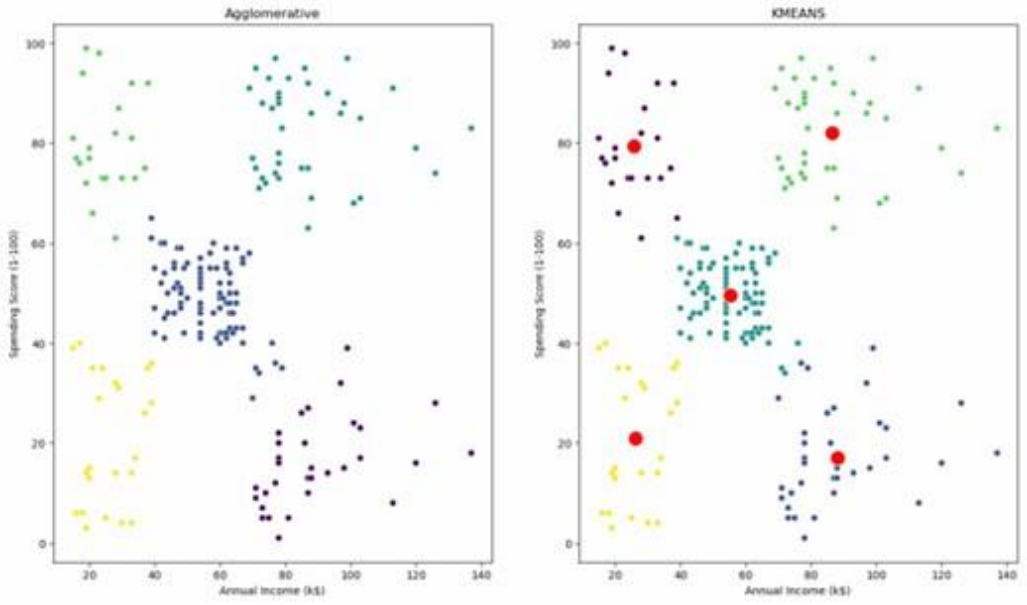
In [27]:
```
alabels
```

Out[27]:
```
array([4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3,
       4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 1,
       4, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 2, 1, 2, 0, 2, 0, 2,
       1, 2, 0, 2, 0, 2, 0, 2, 0, 2, 1, 2, 0, 2, 1, 2, 0, 2, 0, 2, 0, 2,
       0, 2, 0, 2, 0, 2, 1, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2,
       0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2,
       0, 2], dtype=int64)
```

In [28]:
```
plt.figure(figsize=(16,9))
plt.subplot(1,2,1)
plt.title('Agglomerative')
sns.scatterplot(x=x['Annual Income (k$)'],y=x['Spending Score (1-100)'], c=

plt.subplot(1,2,2)
plt.title('KMEANS')
sns.scatterplot(x=x['Annual Income (k$)'],y=x['Spending Score (1-100)'],c=l
sns.scatterplot(cent[:,0],cent[:,1], s=200, color='red')
```

```
C:\Users\admin\anaconda3\lib\site-packages\seaborn\_decorators.py:36: Futu
reWarning: Pass the following variables as keyword args: x, y. From versio
n 0.12, the only valid positional argument will be `data`, and passing oth
er arguments without an explicit keyword will result in an error or misint
erpretation.
  warnings.warn(
```

Out[28]:
```
<AxesSubplot:title={'center':'KMEANS'}, xlabel='Annual Income (k$)', ylabe
l='Spending Score (1-100)'>
```



In [ ]:

Assignment No -4

Title: Association Rule Learning

Problem Statement:
Download Market Basket Optimization dataset from below link.
Data Set: https://www.kaggle.com/hemanthkumar05/market-basket-optimization
This dataset comprises the list of transactions of a retail company over the period of one week. It contains a total of 7501 transaction records where each record consists of the list of items sold in one transaction. Using this record of transactions and items in each transaction, find the association rules between items.
There is no header in the dataset and the first row contains the first transaction, so mentioned header = None here while loading dataset.

a. Follow following steps :
b. Data Preprocessing
c. Generate the list of transactions from the dataset
d. Train Apriori algorithm on the dataset
e. Visualize the list of rules
Generated rules depend on the values of hyper parameters. By increasing the minimum confidence value and find the rules accordingly

Objective:
This assignment will help the students to understand and implement Apriori Algorithm.

S/W Packages and H/W apparatus used:
Linux OS: Ubantu/Windows , Jupyter notebook.
PC with the configuration as Pentium IV 1.7 GHz. 128M.B RAM, 40 G.B HDD, 15""Color Monitor, Keyboard, Mouse

References:

1. Ethem Alpaydin, Introduction to Machine Learning, PHI 2nd Edition,2013.
2. Peter Flach: Machine Learning: The Art and Science of Algorithms that Make Sense of Data,Cambridge University Press, Edition 2012.

Theory:

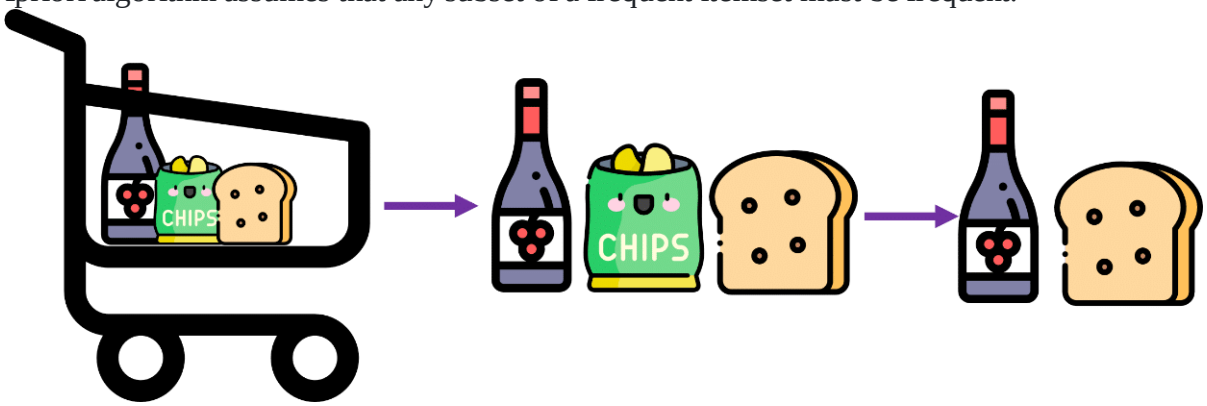
the Apriori algorithm is used for the purpose of association rule mining.Association rule mining is a technique to identify frequent patterns and associations among a set of items.

For example, understanding customer buying habits. By finding correlations and associations between different items that customers place in their „shopping basket," recurring patterns can be derived.

This process of identifying an association between products/items is called association rule mining. To implement association rule mining, many algorithms have been developed. Apriori algorithm is one of the most popular and arguably the most efficient algorithms among them.

Apriori Algorithm

Apriori algorithm assumes that any subset of a frequent itemset must be frequent.



Say, a transaction containing {wine, chips, bread} also contains {wine, bread}. So, according to the principle of Apriori, if {wine, chips, bread} is frequent, then {wine, bread} must also be frequent.

The key concept in the Apriori algorithm is that it assumes all subsets of a frequent itemset to be frequent. Similarly, for any infrequent itemset, all its supersets must also be infrequent.

Here is a dataset consisting of six transactions in an hour. Each transaction is a combination of 0s and 1s, where 0 represents the absence of an item and 1 represents the presence of it.

Here is a dataset consisting of six transactions in an hour. Each transaction is a combination of 0s and 1s, where 0 represents the absence of an item and 1 represents the presence of it.

Transaction ID	Wine	Chips	Bread	Milk

1	1	1	1	1
2	1	0	1	1
3	0	0	1	1
4	0	1	0	0
5	1	1	1	1
6	1	1	0	1

We can find multiple rules from this scenario. For example, in a transaction of wine, chips, and bread, if wine and chips are bought, then customers also buy bread.

{wine, chips} =>; {bread}

In order to select the interesting rules out of multiple possible rules from this small business scenario, we will be using the following measures:

- Support
- Confidence
- List

Support of item *x* is nothing but the ratio of the number of transactions in which item *x* appears to the total number of transactions.

i.e.,

$$\text{Support(wine)} = \frac{\textit{Number of transactions in which the item wine appears}}{\textit{Total number of transactions}}$$

Support(wine) = 4/6= 0.66667

Confidence

Confidence (*x* => *y*) signifies the likelihood of the item *y* being purchased when item *x* is purchased. This method takes into account the popularity of item *x*.

i.e.,

$$\text{Conf(\{wine, chips\} => \{bread\})} = \frac{support(wine,chips,bread)}{support(wine,chips)}$$

$$\frac{4/6}{3/6} = \frac{2}{3} :$$

Conf({wine, chips} => {bread})= = 0.667

Lift

Lift (x => y) is nothing but the „interestingness" or the likelihood of the item y being purchased when item x is sold. Unlike confidence (x => y), this method takes into account

i.e.,

$$\text{lift } (\{\text{wine, chips}\} \Rightarrow \{\text{bread}\}) = \frac{support(wine,chips,bread)}{support(wine,chips)}$$

$$\text{lift } (\{\text{wine, chips}\} \Rightarrow \{\text{bread}\}) = \frac{\frac{2}{6}}{\frac{3}{6} * \frac{4}{6}} = 1$$

- Lift (x => y) = 1 means that there is no correlation within the itemset.
- Lift (x => y) > 1 means that there is a positive correlation within the itemset, i.e., products in the itemset, x and y, are more likely to be bought together.
- Lift (x => y) < 1 means that there is a negative correlation within the itemset, i.e., products in itemset, x and y, are unlikely to be bought together.

Dataset

Below is the transaction data from Day 1. This dataset contains 6 items and 22 transaction records.

	A	B	C	D	E	F
1	Wine	Chips	Bread	Butter	Milk	Apple
2	Wine		Bread	Butter	Milk	
3			Bread	Butter	Milk	
4		Chips				Apple
5	Wine	Chips	Bread	Butter	Milk	Apple
6	Wine	Chips			Milk	
7	Wine	Chips	Bread	Butter		Apple
8	Wine	Chips			Milk	
9	Wine		Bread			Apple
10	Wine		Bread	Butter	Milk	
11		Chips	Bread	Butter		Apple
12	Wine			Butter	Milk	Apple
13	Wine	Chips	Bread	Butter	Milk	
14	Wine		Bread		Milk	Apple
15	Wine		Bread	Butter	Milk	Apple
16	Wine	Chips	Bread	Butter	Milk	Apple
17		Chips	Bread	Butter	Milk	Apple
18		Chips		Butter	Milk	Apple
19	Wine	Chips	Bread	Butter	Milk	Apple
20	Wine		Bread	Butter	Milk	Apple
21	Wine	Chips	Bread		Milk	Apple
22		Chips				

Environment Setup:

Before we move forward, we need to install the „apyori“ package first.

```
pip install apyori
```

Market Basket Analysis Implementation within Python

With the help of the apyori package, we will be implementing the Apriori algorithm in order to help the manager in market basket analysis.

Step 1: Import the libraries

```
In [1]: #Importing the required datasets
        import numpy as np
        import pandas as pd
        from apyori import apriori
```

Step 2: Load the dataset

```
In [2]: #Loading the dataset
        store_data = pd.read_csv('Day1.csv', header=None)
```

Step 3: Have a glance at the records

```
In [3]: #Having a glance at the records
        store_data
```

Out[3]:

	0	1	2	3	4	5
0	Wine	Chips	Bread	Butter	Milk	Apple
1	Wine	NaN	Bread	Butter	Milk	NaN
2	NaN	NaN	Bread	Butter	Milk	NaN
3	NaN	Chips	NaN	NaN	NaN	Apple
4	Wine	Chips	Bread	Butter	Milk	Apple
5	Wine	Chips	NaN	NaN	Milk	NaN
6	Wine	Chips	Bread	Butter	NaN	Apple
7	Wine	Chips	NaN	NaN	Milk	NaN
8	Wine	NaN	Bread	NaN	NaN	Apple
9	Wine	NaN	Bread	Butter	Milk	NaN
10	NaN	Chips	Bread	Butter	NaN	Apple
11	Wine	NaN	NaN	Butter	Milk	Apple
12	Wine	Chips	Bread	Butter	Milk	NaN
13	Wine	NaN	Bread	NaN	Milk	Apple
14	Wine	NaN	Bread	Butter	Milk	Apple
15	Wine	Chips	Bread	Butter	Milk	Apple
16	NaN	Chips	Bread	Butter	Milk	Apple
17	NaN	Chips	NaN	Butter	Milk	Apple
18	Wine	Chips	Bread	Butter	Milk	Apple
19	Wine	NaN	Bread	Butter	Milk	Apple
20	Wine	Chips	Bread	NaN	Milk	Apple
21	NaN	Chips	NaN	NaN	NaN	NaN

Step 4 :Convert Pandas DataFrame into a list of lists

```
In [5]: #Converting the pandas dataframe into a list of lists
        records = []
        for i in range(0, 22):
            records.append([str(store_data.values[i,j]) for j in range(0, 6)])
```

Step 5: Build the Apriori model

```
In [7]: #Building the first apriori model
        association_rules = apriori(records, min_support=0.50, min_confidence=0.7, min_lift=1.2, min_length=2)
        association_results = list(association_rules)
```

Step 6: Print out the number of rules

```
In [8]: #Getting the number of rules
        print(len(association_results))
```

Step 7: Have a glance at the rule

```
In [10]: #Glancing at the first rule
         print(association_results)
```

```
[RelationRecord(items=frozenset({'Milk', 'Butter', 'Bread'}), support=0.5, ordered_statistics=[OrderedStatistic(items_base=froz
enset({'Milk', 'Bread'}), items_add=frozenset({'Butter'}), confidence=0.8461538461538461, lift=1.241025641025641)])]
```

The support value for the first rule is 0.5. This number is calculated by dividing the number of transactions containing „Milk,“ „Bread,“ and „Butter“ by the total number of transactions.

The confidence level for the rule is 0.846, which shows that out of all the transactions that contain both “Milk” and “Bread”, 84.6 % contain „Butter“ too.

The lift of 1.241 tells us that „Butter" is 1.241 times more likely to be bought by the customers who buy both „Milk" and „Butter" compared to the default likelihood sale of „Butter."

In [2]:
```
!pip install mlxtend
```

```
Collecting mlxtend
  Downloading mlxtend-0.23.1-py3-none-any.whl (1.4 MB)
     ---------------------------------------- 1.4/1.4 MB 1.7 MB/s eta 0:00:00
Requirement already satisfied: matplotlib>=3.0.0 in c:\users\admin\anaconda3\l
ib\site-packages (from mlxtend) (3.5.2)
Requirement already satisfied: joblib>=0.13.2 in c:\users\admin\anaconda3\lib
\site-packages (from mlxtend) (1.1.0)
Requirement already satisfied: scikit-learn>=1.0.2 in c:\users\admin\anaconda3
\lib\site-packages (from mlxtend) (1.0.2)
Requirement already satisfied: scipy>=1.2.1 in c:\users\admin\anaconda3\lib\si
te-packages (from mlxtend) (1.9.1)
Requirement already satisfied: numpy>=1.16.2 in c:\users\admin\anaconda3\lib\s
ite-packages (from mlxtend) (1.21.5)
Requirement already satisfied: pandas>=0.24.2 in c:\users\admin\anaconda3\lib
\site-packages (from mlxtend) (1.4.4)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\admin\anaconda3\l
ib\site-packages (from matplotlib>=3.0.0->mlxtend) (1.4.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\admin\anaconda
3\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (2.8.2)
Requirement already satisfied: pillow>=6.2.0 in c:\users\admin\anaconda3\lib\s
ite-packages (from matplotlib>=3.0.0->mlxtend) (9.2.0)
Requirement already satisfied: pyparsing>=2.2.1 in c:\users\admin\anaconda3\li
b\site-packages (from matplotlib>=3.0.0->mlxtend) (3.0.9)
Requirement already satisfied: cycler>=0.10 in c:\users\admin\anaconda3\lib\si
te-packages (from matplotlib>=3.0.0->mlxtend) (0.11.0)
Requirement already satisfied: packaging>=20.0 in c:\users\admin\anaconda3\lib
\site-packages (from matplotlib>=3.0.0->mlxtend) (21.3)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\admin\anaconda3\l
ib\site-packages (from matplotlib>=3.0.0->mlxtend) (4.25.0)
Requirement already satisfied: pytz>=2020.1 in c:\users\admin\anaconda3\lib\si
te-packages (from pandas>=0.24.2->mlxtend) (2022.1)
Requirement already satisfied: threadpoolctl>=2.0.0 in c:\users\admin\anaconda
3\lib\site-packages (from scikit-learn>=1.0.2->mlxtend) (2.2.0)
Requirement already satisfied: six>=1.5 in c:\users\admin\anaconda3\lib\site-p
ackages (from python-dateutil>=2.7->matplotlib>=3.0.0->mlxtend) (1.16.0)
Installing collected packages: mlxtend
Successfully installed mlxtend-0.23.1
```

In [20]:
```
import csv
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
import pandas as pd
import numpy as np
```

In [8]:
```
data=[]
with open('../admin/Desktop/TE IT/Market_Basket_Optimisation.csv') as file:
    reader = csv.reader(file, delimiter=',')
    for row in reader:
        data +=[row]
```

```
In [9]: data[1:10] #list of list

Out[9]: [['burgers', 'meatballs', 'eggs'],
         ['chutney'],
         ['turkey', 'avocado'],
         ['mineral water', 'milk', 'energy bar', 'whole wheat rice', 'green tea'],
         ['low fat yogurt'],
         ['whole wheat pasta', 'french fries'],
         ['soup', 'light cream', 'shallot'],
         ['frozen vegetables', 'spaghetti', 'green tea'],
         ['french fries']]

In [10]: len(data)

Out[10]: 7501

In [11]: te = TransactionEncoder()
         x = te.fit_transform(data)

In [12]: x

Out[12]: array([[False,  True,  True, ...,  True, False, False],
                [False, False, False, ..., False, False, False],
                [False, False, False, ..., False, False, False],
                ...,
                [False, False, False, ..., False, False, False],
                [False, False, False, ..., False, False, False],
                [False, False, False, ..., False,  True, False]])

In [13]: te.columns_
          'sparkling water',
          'spinach',
          'strawberries',
          'strong cheese',
          'tea',
          'tomato juice',
          'tomato sauce',
          'tomatoes',
          'toothpaste',
          'turkey',
          'vegetables mix',
          'water spray',
          'white wine',
          'whole weat flour',
          'whole wheat pasta',
          'whole wheat rice',
          'yams',
          'yogurt cake',
          'zucchini']
```

```
In [21]: df = pd.DataFrame(x, columns=te.columns_)
```



```
In [22]: df
```

Out[22]:

	asparagus	almonds	antioxydant juice	asparagus	avocado	babies food	bacon	barbecue sauce	black tea	bl
0	False	True	True	False	True	False	False	False	False	
1	False	False	False	False	False	False	False	False	False	
2	False	False	False	False	False	False	False	False	False	
3	False	False	False	False	True	False	False	False	False	
4	False	False	False	False	False	False	False	False	False	
...	...	...	...	...	...	...	...	...	...	
7496	False	False	False	False	False	False	False	False	False	
7497	False	False	False	False	False	False	False	False	False	
7498	False	False	False	False	False	False	False	False	False	
7499	False	False	False	False	False	False	False	False	False	
7500	False	False	False	False	False	False	False	False	False	

7501 rows × 120 columns

```
In [23]: # Finding frequent itemsets
freq_itemset = apriori(df, min_support=0.01, use_colnames=True)
```

```
In [24]: freq_itemset
```

Out[24]:

	support	itemsets
0	0.020397	(almonds)
1	0.033329	(avocado)
2	0.010799	(barbecue sauce)
3	0.014265	(black tea)
4	0.011465	(body spray)
...	...	...
252	0.011065	(milk, ground beef, mineral water)
253	0.017064	(spaghetti, ground beef, mineral water)
254	0.015731	(milk, spaghetti, mineral water)
255	0.010265	(spaghetti, mineral water, olive oil)
256	0.011465	(pancakes, spaghetti, mineral water)

257 rows × 2 columns

In [25]:
```
#Find the rules
rules = association_rules(freq_itemset, metric='confidence', min_threshold=0.10
```


In [26]:
```
rules = rules[['antecedents','consequents','support','confidence']]
rules
```

Out[26]:

	antecedents	consequents	support	confidence
0	(avocado)	(mineral water)	0.011598	0.348000
1	(cake)	(burgers)	0.011465	0.141447
2	(burgers)	(cake)	0.011465	0.131498
3	(burgers)	(chocolate)	0.017064	0.195719
4	(chocolate)	(burgers)	0.017064	0.104150
...	...	...	...	...
315	(olive oil)	(spaghetti, mineral water)	0.010265	0.155870
316	(pancakes, spaghetti)	(mineral water)	0.011465	0.455026

...	...	...	...	...
315	(olive oil)	(spaghetti, mineral water)	0.010265	0.155870
316	(pancakes, spaghetti)	(mineral water)	0.011465	0.455026
317	(pancakes, mineral water)	(spaghetti)	0.011465	0.339921
318	(spaghetti, mineral water)	(pancakes)	0.011465	0.191964
319	(pancakes)	(spaghetti, mineral water)	0.011465	0.120617

320 rows × 4 columns

In [27]:
```
rules[rules['antecedents'] == {'cake'}]['consequents']
```

Out[27]:
```
1               (burgers)
25            (chocolate)
27                 (eggs)
29          (french fries)
30    (frozen vegetables)
32            (green tea)
35                 (milk)
36        (mineral water)
39             (pancakes)
40            (spaghetti)
Name: consequents, dtype: object
```

In [ ]:
```

```

Conclusion:

Apriori algorithm is implemented

PART B:

Advanced Database Management Systems

Assignment: 1

AIM:

Create a database with suitable example using MongoDB and implement

1. Inserting and saving document (batch insert, insert validation)
2. Removing document
3. Updating document (document replacement, using modifiers, up inserts, updating multiple documents, returning updated documents)
4. Execute at least 10 queries on any suitable MongoDB database that demonstrates following:
    - Find and find One (specific values)
    - Query criteria (Query conditionals, OR queries, $not, Conditional semantics)
    - Type-specific queries (Null, Regular expression, Querying arrays) where queries
    - Cursors (Limit, skip, sort, advanced query options)

PROBLEM STATEMENT /DEFINITION

Create a database with suitable example using MongoDB and implement

- Inserting and saving document (batch insert, insert validation)
- Removing document
- Updating document (document replacement, using modifiers, upserts, updating multiple documents, returning updated documents)

OBJECTIVE:

To understand Mongodb basic commands
To implement the concept of document oriented databases.
To understand Mongodb retrieval commands

THEORY:

SQL VsMongoDB

SQL Concepts	MongoDB Concepts
Database	Database
Table	Collection
Row	Document 0r BSON Document
Column	Field
Index	Index
Table Join	Embedded Documents & Linking
Primary Key	Primary Key
Specify Any Unique Column Or Column Combination As Primary Key.	In Mongodb, The Primary Key Is Automatically Set To The _Id Field.
Aggregation (E.G. Group By)	Aggregation Pipeline

1. Create a collection in mongodb

```
db.createCollection(“Teacher_info")
```

2. Create a capped collection in mongodb

```
>db.createCollection("audit", {capped:true, size:20480})
{ "ok" : 1 }
```

3. Insert a document into collection

```
db.Teacher_info.insert( { Teacher_id: “Pic001", Teacher_Name: “Ravi",Dept_Name: “IT”, Sal:30000, status: "A" } )

db.Teacher_info.insert( { Teacher_id: “Pic002", Teacher_Name: “Ravi",Dept_Name: “IT”, Sal:20000, status: "A" } )

db.Teacher_info.insert( { Teacher_id: “Pic003", Teacher_Name: “Akshay",Dept_Name: “Comp”, Sal:25000, status: “N" } )
```

4. Update a document into collection

db. Teacher_info.update({ sal: { $gt: 25000 } }, { $set: { Dept_name: “ETC" } }, { multi: true })

db. Teacher_info.update({ status: "A" } , { $inc: { sal: 10000 } }, { multi: true })

5. Remove a document from collection

db.Teacher_info.remove({Teacher_id: "pic001"}); db. Teacher_info.remove({})

6. Alter a field into a mongodb document

db.Teacher_info.update({ }, { $set: { join_date: new Date() } }, { multi: true})

7. To drop a particular collection

db.Teacher_info.drop()

Retrieval From Database:-

When we retrieve a document from mongodb collection it always add a _id field in the every document which conatin unique _id field.

ObjectId(<hexadecimal>)

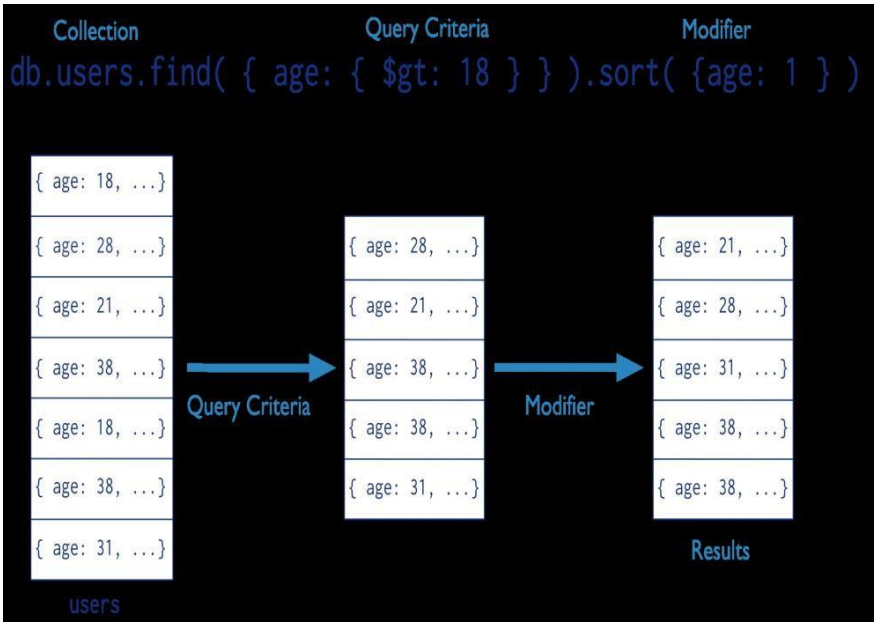
Returns a new ObjectId value. The 12-byte ObjectId value consists of: 4-byte

value representing the seconds since the Unix epoch,

3-byte machine identifier,

2-byte process id, and

3-byte counter,starting with a random value.



1.Retrieve a collection in mongodb using Find command

db.Teacher.find()

{ "_id" : 101, "Name" : "Dev",
"Address" : [{ "City" : "Pune", "Pin" : 444043 }],
"Department" : [{ "Dept_id" : 111, "Dept_name" : "IT" }], "Salary" : 78000 }

{ "_id" : 135, "Name" : "Jennifer", "Address" : [{ "City" :
"Mumbai", "Pin" : 444111 }], "Department" : [{ "Dept_id" : 112, "Dept_name" :
"COMP" }], "Salary" : 65000 }
{ "_id" : 126, "Name" : "Gaurav", "Address" : [{ "City" : "Nashik",
"Pin" : 444198 }], "Department" : [{ "Dept_id" : 112, "Dept_name"
: "COMP" }], "Salary" : 90000 }
{ "_id" : 175, "Name" : "Shree", "Address" : [{ "City" : "Nagpur",
"Pin" : 444158 }], "Department" : [{ "Dept_id" : 113, "Dept_name"
: "ENTC" }], "Salary" : 42000 }
{ "_id" : 587, "Name" : "Raman", "Address" : [{ "City" : "Banglore",
"Pin" : 445754 }], "Department" : [{ "Dept_id" : 113, "Dept_name"
: "ENTC" }], "Salary" : 79000 }
{ "_id" : 674, "Name" : "Mandar", "Address" : [{ "City" : "Jalgaon",
"Pin" : 465487 }], "Department" : [{ "Dept_id" : 111, "Dept_name"
: "IT" }], "Salary" : 88000 }
{ "_id" : 573, "Name" : "Manish", "Address" : [{ "City" : "Washim",
"Pin" : 547353 }], "Department" : [{ "Dept_id" : 112, "Dept_name"
: "COMP" }], "Salary" : 65000 }

1. Retrieve a document from collection in mongodb using Find command using condition

>db.Teacher_info.find({sal: 25000})

2. Retrieve a document from collection in mongodb using Find command using or operator

>db.Teacher_info.find({ $or: [{ status: "A" } , { sal:50000 }] })

3. Retrieve a document from collection in mongodb using Find command using greater than , less than, greater than and equal to ,less than and equal to operator

>db. Teacher_info.find({ sal: **{ $gt:** 40000 } })

>db.media.find({ Released : **{$gt :** 2000} }, { **"Cast" : 0** })

{ "_id" : ObjectId("4c4369a3c603000000007ed3"), "Type" : "DVD", "Title" :"Toy Story 3", "Released" : 2010 }

>db.media.find ({ Released : **{$gte** : 1999 } }, { "Cast" : 0 })

{ "_id" : ObjectId("4c43694bc603000000007ed1"), "Type" : "DVD", "Title" :"Matrix, The", "Released" : 1999 }

{ "_id" : ObjectId("4c4369a3c603000000007ed3"), "Type" : "DVD", "Title" :"Toy Story 3", "Released" : 2010 }

>db.media.find ({ Released : **{$lt :** 1999 } }, { "Cast" : 0 })

{ "_id" : ObjectId("4c436969c603000000007ed2"), "Type" : "DVD", "Title" : "Blade Runner", "Released" : 1982 }

>db.media.find({Released : **{$lte:** 1999}}, { "Cast" : 0 })

{ "_id" : ObjectId("4c43694bc603000000007ed1"), "Type" : "DVD", "Title" :"Matrix, The", "Released" : 1999 }

{ "_id" : ObjectId("4c436969c603000000007ed2"), "Type" : "DVD", "Title" :"Blade Runner", "Released" : 1982 }

>db.media.find({Released : **{$gte: 1990, $lt : 2010**}}, { "Cast" : 0 })

{ "_id" : ObjectId("4c43694bc603000000007ed1"), "Type" : "DVD", "Title" :"Matrix, The", "Released" : 1999 }

Retrieval a value from document which contain array field Exact Match

on an Array

db.inventory.find({ tags: ['fruit', 'food', 'citrus'] })

Match an Array Element db.inventory.find({

tags: 'fruit' }) **Match a Specific Element of an**

Array db.inventory.find({ 'tags.**0**' : 'fruit' })

6. MongoDB provides a db.collection.findOne() method as a special case of find() that returns a single document.

7. Exclude One Field from a Result Set

>db.records.Find({ "user_id": { $lt: 42} }, { history: 0})

8. Return Two fields and the _id Field

>db.records.find({ "user_id": { $lt: 42} }, { "name": 1, "email": 1})

9. Return Two Fields and Exclude _id

>db.records.find({ "user_id": { $lt: 42} }, { "_id": 0, "name": 1 , "email": 1 })

10. Retrieve a collection in mongodb using Find command and pretty appearance

>db.<collection>.find().pretty()

db.users.find(collection{age:{$gt:18}},querycriteria{name :1,address:1}projection

Retrieve a document in ascending or descending order using 1 for ascending and -1 for descendingfrom collection in mongodb

>db. Teacher_info.find({ status: "A" }).sort({sal: -1 })

>db.audit.find().sort({ $natural: -1 }).limit (10)

>db.Employee.find().sort({_id:-1})

{ "_id" : 106, "Name" : "RAJ", "Address" : [{ "City" : "NASIK", "Pin" : 411002 }], "Department" : [{ "Dept_id" : 113, "Dept_name" : "ACCOUNTING" }], "Salary" : 50000 }
{ "_id" : 105, "Name" : "ASHOK", "Address" : [{ "City" : "NASIK", "Pin" : 411002 }], "Department" : [{ "Dept_id" : 113, "Dept_name" : "ACCOUNTING" }], "Salary" : 40000 }
{ "_id" : 104, "Name" : "JOY", "Address" : [{ "City" : "Pune", "Pin" : 444043 }], "Department" : [{ "Dept_id" : 112, "Dept_name" : "SALES" }], "Salary" : 20000 }
{ "_id" : 103, "Name" : "RAM", "Address" : [{ "City" : "Pune", "Pin" : 444043 }], "Department" : [{ "Dept_id" : 112, "Dept_name" : "SALES" }], "Salary" : 10000 }
{ "_id" : 102, "Name" : "AKASH", "Address" : [{ "City" : "Pune", "Pin" : 444043 }], "Department" : [{ "Dept_id" : 111, "Dept_name" : "HR" }], "Salary" : 80000 }
{ "_id" : 101, "Name" : "Dev", "Address" : [{ "City" : "Pune", "Pin" : 444043 }], "Department" : [{ "Dept_id" : 111, "Dept_name" : "HR" }], "Salary" : 78000 }

>db.Employee.find().sort({_id:1})

{ "_id" : 101, "Name" : "Dev", "Address" : [{ "City" : "Pune", "Pin" : 444043 }], "Department" : [{ "Dept_id" : 111, "Dept_name" : "HR" }], "Salary" : 78000 }
{ "_id" : 102, "Name" : "AKASH", "Address" : [{ "City" : "Pune", "Pin" : 444043 }], "Department" : [{ "Dept_id" : 111, "Dept_name" : "HR" }], "Salary" : 80000 }

{ "_id" : 103, "Name" : "RAM", "Address" : [{ "City" : "Pune", "Pin" :
444043 }], "Department" : [{ "Dept_id" : 112, "Dept_name" : "SALES" }], "Salary" : 10000 }
{ "_id" : 104, "Name" : "JOY", "Address" : [{ "City" : "Pune", "Pin" :
444043 }], "Department" : [{ "Dept_id" : 112, "Dept_name" : "SALES" }], "Salary" : 20000 }
{ "_id" : 105, "Name" : "ASHOK", "Address" : [{ "City" : "NASIK", "Pin" :
411002 }], "Department" : [{ "Dept_id" : 113, "Dept_name" : "ACCOUNTING"
}], "Salary" : 40000 }
{ "_id" : 106, "Name" : "RAJ", "Address" : [{ "City" : "NASIK", "Pin" :
411002 }], "Department" : [{ "Dept_id" : 113, "Dept_name" : "ACCOUNTING"
}], "Salary" : 50000 }
>db.Employee.find().sort({$natural:-1}).limit(2)
{ "_id" : 106, "Name" : "RAJ", "Address" : [{ "City" : "NASIK", "Pin" :
411002 }], "Department" : [{ "Dept_id" : 113, "Dept_name" : "ACCOUNTING"
}], "Salary" : 50000 }
{ "_id" : 105, "Name" : "ASHOK", "Address" : [{ "City" : "NASIK", "Pin" :
411002 }], "Department" : [{ "Dept_id" : 113, "Dept_name" : "ACCOUNTING"
}], "Salary" : 40000 }

>db.Employee.find().sort({$natural:1}).limit(2)

{ "_id" : 101, "Name" : "Dev", "Address" : [{ "City" : "Pune", "Pin" :
444043 }], "Department" : [{ "Dept_id" : 111, "Dept_name" : "HR" }], "Salary" : 78000 }
{ "_id" : 102, "Name" : "AKASH", "Address" : [{ "City" : "Pune", "Pin" :
444043 }], "Department" : [{ "Dept_id" : 111, "Dept_name" : "HR" }], "Salary" : 80000 }
>db.Employee.find({Salary:{$in:[10000,30000]}})
{ "_id" : 103, "Name" : "RAM", "Address" : [{ "City" : "Pune", "Pin" :
444043 }], "Department" : [{ "Dept_id" : 112, "Dept_name" : "SALES" }], "Salary" : 10000 }
>db.Employee.update({"Name":"RAM"},{ $set :{Address:{City: "Nasik"}}}) WriteResult({
"nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>db.Employee.find({"Name":"RAM"})
{ "_id" : 103, "Name" : "RAM", "Address" : { "City" : "Nasik" },
"Department" : [{ "Dept_id" : 112, "Dept_name" : "SALES" }], "Salary" : 10000 }
>db.Employee.update({"Name":"RAM"},{$inc :{"Salary": 10000 } }) WriteResult({
"nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>db.Employee.find({"Name":"RAM"})
{ "_id" : 103, "Name" : "RAM", "Address" : { "City" : "Nasik" },
"Department" : [{ "Dept_id" : 112, "Dept_name" : "SALES" }], "Salary" : 20000 }

Retrieve documentwith a particular from collection in mongodb

>db.Employee.find().limit(2).pretty()

{
"_id" : 101,

```
        "Name" : "Dev",
        "Address" : [
                {
                        "City" : "Pune", "Pin" :
                        444043
                }
        ],
        "Department" : [
                {
                        "Dept_id" : 111,
                        "Dept_name" : "HR"
                }
        ],
        "Salary" : 78000
}
{
        "_id" : 102, "Name" :
        "AKASH",
          "Address" : [
                {
                        "City" : "Pune", "Pin" :
                        444043
                }
        ],
        "Department" : [
                {
                        "Dept_id" : 111,
                        "Dept_name" : "HR"
                }
        ],
        "Salary" : 80000
}
```


Retrieve document skipping some documents from collection in mongodb

>db.Employee.find().skip(3).pretty()


```
{"_id" : 104, "Name" : "JOY",
          "Address" : [
                {
                        "City" : "Pune", "Pin"
                }
        ],
        "Department" : [{
                        "Dept_id" : 112, "Dept_name" :
                        "SALES"
                }
```

```
}
{
        "_id" : 105, "Name" :
        "ASHOK",
           "Address" : [
                {
                        "City" : "NASIK",
                        "Pin" : 411002
                }
        ],
        "Department" : [
                {
                        "Dept_id" : 113, "Dept_name" :
                        "ACCOUNTING"
                }
        ],
        "Salary" : 40000
}
{
        "_id" : 106, "Name" :
        "RAJ",
           "Address" : [
                {
                        "City" : "NASIK",
                        "Pin" : 411002
                }
        ],
        "Department" : [
                {
                        "Dept_id" : 113, "Dept_name" :
                        "ACCOUNTING"
                }
        ],
        "Salary" : 50000
}
```


REFERENCE BOOK:

Kristina Chodorow, MongoDB The definitive guide, O‟Reilly Publications, ISBN:978-93-5110-269-4,2nd Edition.

Title: Create a database with suitable example using MongoDB and implement CURD operations, Query Operations and Cursor operations on it.

1. CURD Operations:

1) Display list of existing databases

```
> show dbs
admin    0.000GB
assign1  0.000GB
config   0.000GB
local    0.000GB
```

2) Switch database

```
> use assign1
switched to db assign1
```

3) Create Collection

```
> db.createCollection("Book");
{ "ok" : 1 }
```

4) Display list of existing collections

```
> show collections
Book
book
```

5) Insert documents into collection

```
> db.Book.insert({id:"3586824624",title:"Database Management
System",author:"S.Sudharshan",dept:"IT",subject:"ADBMS",price:756.76})
WriteResult({ "nInserted" : 1 })
> db.Book.insert({id:"8753698428",title:"Designing The User Interface",author:"Alan
Dix",dept:"IT",subject:"HCI",price:876.75})
WriteResult({ "nInserted" : 1 })
```

6) Retrieve and Display Documents

```
> db.Book.find().pretty()
{
        "_id" : ObjectId("64ecdfb6848ae0682d696f51"),
        "id" : "3586824624",
        "title" : "Database Management System",
        "author" : "S.Sudharshan",
        "dept" : "IT",
        "subject" : "ADBMS",
        "price" : 756.76
}
{
        "_id" : ObjectId("64ece032848ae0682d696f52"),
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 876.75
}
```

7) Bulk insert

```
> db.Book.insert([{id:"2376574365",title:"Machine Design",author:"Saul
          Fenster",dept:"ME",subject:"Machine
          Design",price:975},{id:"4868362945",title:"Digital
          Electronics",dept:"EE",price:832.765},{id:298374574,title:"Engineering
          Thermodyanamics",author:"P.K.Nag",dept:"ME",price:732.98}])
BulkWriteResult({
        "writeErrors" : [ ],
        "writeConcernErrors" : [ ],
        "nInserted" : 3,
        "nUpserted" : 0,
        "nMatched" : 0,
        "nModified" : 0,
        "nRemoved" : 0,
        "upserted" : [ ]
})
```

```
> db.Book.find().pretty()
        {
         "_id" : ObjectId("64ece032848ae0682d696f52"),
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 876.75
        }
        {
        "_id" : ObjectId("64eceb95caf728f8ddcefd13"),
        "id" : "2376574365",
        "title" : "Machine Design",
        "author" : "Saul Fenster",
        "dept" : "ME",
        "subject" : "Machine Design",
        "price" : 975
        }
        {
        "_id" : ObjectId("64eceb95caf728f8ddcefd14"),
        "id" : "4868362945",
        "title" : "Digital Electronics",
        "dept" : "EE",
        "price" : 832.765
        }
        {
        "_id" : ObjectId("64eceb95caf728f8ddcefd15"),
        "id" : 298374574,
        "title" : "Engineering Thermodyanamics",
        "author" : "P.K.Nag",
        "dept" : "ME",
        "price" : 732.98
        }
```

8) Delete particular document

```
> db.Book.remove({title:"Engineering Thermodyanamics"})
WriteResult({ "nRemoved" : 1 })
> db.Book.find().pretty()
{
```

```
  "_id" : ObjectId("64ecdfb6848ae0682d696f51"),
        "id" : "3586824624",
        "title" : "Database Management System",
        "author" : "S.Sudharshan",
        "dept" : "IT",
        "subject" : "ADBMS",
        "price" : 756.76
}
{
        "_id" : ObjectId("64ece032848ae0682d696f52"),
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 876.75
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd13"),
        "id" : "2376574365",
        "title" : "Machine Design",
        "author" : "Saul Fenster",
        "dept" : "ME",
        "subject" : "Machine Design",
        "price" : 975
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd14"),
        "id" : "4868362945",
        "title" : "Digital Electronics",
        "dept" : "EE",
        "price" : 832.76
```

9) Update particular document

```
> db.Book.update({title:"Machine Design"},{id:"23567897656",title:"Machine
        Design",author:"Alexander Fleming",subject:"Machine Design",price:823.098})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
> db.Book.find(author:"Alexander  Fleming").pretty()
uncaught exception: SyntaxError: missing ) after argument list :
@(shell):1:19
> db.Book.find().pretty()
{
      "_id" : ObjectId("64ecdfb6848ae0682d696f51"),
      "id" : "3586824624",
      "title" : "Database Management System",
      "author" : "S.Sudharshan",
      "dept" : "IT",
      "subject" : "ADBMS",
      "price" : 756.76
}
{
      "_id" : ObjectId("64ece032848ae0682d696f52"),
      "id" : "8753698428",
      "title" : "Designing The User Interface",
      "author" : "Alan Dix",
      "dept" : "IT",
      "subject" : "HCI",
      "price" : 876.75
}
{
      "_id" : ObjectId("64eceb95caf728f8ddcefd13"),
      "id" : "23567897656",
      "title" : "Machine Design",
      "author" : "Alexander Fleming",
      "subject" : "Machine Design",
      "price" : 823.098
}
{
      "_id" : ObjectId("64eceb95caf728f8ddcefd14"),
      "id" : "4868362945",
      "title" : "Digital Electronics",
      "dept" : "EE",
```

```
"price" : 832.765
}
> db.Book.update({title:"Digital Electronics"},{$set:{subject:"DSP"}})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
> db.Book.find().pretty()
{
        "_id" : ObjectId("64ecdfb6848ae0682d696f51"),
        "id" : "3586824624",
        "title" : "Database Management System",
        "author" : "S.Sudharshan",
        "dept" : "IT",
        "subject" : "ADBMS",
        "price" : 756.76
}
{
        "_id" : ObjectId("64ece032848ae0682d696f52"),
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 876.75
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd13"),
        "id" : "23567897656",
        "title" : "Machine Design",
        "author" : "Alexander Fleming",
        "subject" : "Machine Design",
        "price" : 823.098
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd14"),
        "id" : "4868362945",
        "title" : "Digital Electronics",
        "dept" : "EE",
        "price" : 832.765,
        "subject" : "DSP"
}
```

2. Query Operations:

1) Display documents with value of an attribute between given range

```
> db.Book.find({price:{$gte:700,$lte:800}}).pretty()
{
        "_id" : ObjectId("64ecdfb6848ae0682d696f51"),
        "id" : "3586824624",
        "title" : "Database Management System",
        "author" : "S.Sudharshan",
        "dept" : "IT",
        "subject" : "ADBMS",
        "price" : 756.76
}
```

2) Display documents with attribute value greater than given value

```
> db.Book.find({price:{$gte:800}}).pretty()
{
        "_id" : ObjectId("64ece032848ae0682d696f52"),
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 876.75
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd13"),
        "id" : "23567897656",
        "title" : "Machine Design",
        "author" : "Alexander Fleming",
        "subject" : "Machine Design",
        "price" : 823.098
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd14"),
        "id" : "4868362945",
        "title" : "Digital Electronics",
        "dept" : "EE",
        "price" : 832.765,
```

```
    "subject" : "DSP"
}
```

3) Count operation with condition

```
> db.Book.find({price:{$gte:800}}).count()
3
```

4) Display value of only specific attribute

```
> db.Book.find({},{_id:0,title:1})
{ "title" : "Database Management System" }
{ "title" : "Designing The User Interface" }
{ "title" : "Machine Design" }
{ "title" : "Digital Electronics" }
```

5) Display all documents without _id

```
> db.Book.find({},{_id:false}).pretty()
{
        "id" : "3586824624",
        "title" : "Database Management System",
        "author" : "S.Sudharshan",
        "dept" : "IT",
        "subject" : "ADBMS",
        "price" : 756.76
}
{
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 876.75
}
{
        "id" : "23567897656",
        "title" : "Machine Design",
        "author" : "Alexander Fleming",
        "subject" : "Machine Design",
```

```
                "price" : 823.098
        }
{
                "id" : "4868362945",
                "title" : "Digital Electronics",
                "dept" : "EE",
                "price" : 832.765,
                "subject" : "DSP"
        }
```


7) Display documents with name starting/ending with given letter


```
> db.Book.find({author:/^A/}).pretty()
{
        "_id" : ObjectId("64ece032848ae0682d696f52"),
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 876.75
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd13"),
        "id" : "23567897656",
        "title" : "Machine Design",
        "author" : "Alexander Fleming",
        "subject" : "Machine Design",
        "price" : 823.098
}
> db.Book.find({author:/x$/}).pretty()
{
        "_id" : ObjectId("64ece032848ae0682d696f52"),
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 876.75
```

```
}
```

8) Display distinct values on any attribute

```
> db.Book.distinct("dept")
[ "EE", "IT" ]
```

9) Display only first document having given value of an attribute

```
> db.Book.findOne({dept:"IT"})
{
        "_id" : ObjectId("64ecdfb6848ae0682d696f51"),
        "id" : "3586824624",
        "title" : "Database Management System",
        "author" : "S.Sudharshan",
        "dept" : "IT",
        "subject" : "ADBMS",
        "price" : 756.76
}
```

10) Decrement value of an attribute by given value (also include condition)

```
> db.Book.update({id:"3586824624"},{$inc:{price:50}})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
> db.Book.find().pretty()
{
        "_id" : ObjectId("64ecdfb6848ae0682d696f51"),
        "id" : "3586824624",
        "title" : "Database Management System",
        "author" : "S.Sudharshan",
        "dept" : "IT",
        "subject" : "ADBMS",
        "price" : 806.76
}
{
        "_id" : ObjectId("64ece032848ae0682d696f52"),
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
```

```
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 876.75


{
        "_id" : ObjectId("64eceb95caf728f8ddcefd13"),
        "id" : "23567897656",
        "title" : "Machine Design",
        "author" : "Alexander Fleming",
        "subject" : "Machine Design",
        "price" : 823.098
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd14"),
        "id" : "4868362945",
        "title" : "Digital Electronics",
        "dept" : "EE",
        "price" : 832.765,
        "subject" : "DSP"
}
```

11) AND operation

```
> db.Book.find({$and:[{author:"Alan  Dix"},{dept:"IT"}]}).pretty()
{
        "_id" : ObjectId("64ece032848ae0682d696f52"),
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 876.75
}
```

12) OR operation

```
> db.Book.find({$or:[{author:"Alan  Dix"},{dept:"IT"}]}).pretty()
{
        "_id" : ObjectId("64ecdfb6848ae0682d696f51"),
        "id" : "3586824624",
        "title" : "Database Management System",
```

```
        "author" : "S.Sudharshan",
        "dept" : "IT",
        "subject" : "ADBMS",
        "price" : 806.76
}

{
        "_id" : ObjectId("64ece032848ae0682d696f52"),
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 876.75
}
```


13) NOT operation

```
> db.Book.find({author:{$ne:"Alan  Dix"}}).pretty()
{
        "_id" : ObjectId("64ecdfb6848ae0682d696f51"),
        "id" : "3586824624",
        "title" : "Database Management System",
        "author" : "S.Sudharshan",
        "dept" : "IT",
        "subject" : "ADBMS",
        "price" : 806.76
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd13"),
        "id" : "23567897656",
        "title" : "Machine Design",
        "author" : "Alexander Fleming",
        "subject" : "Machine Design",
        "price" : 823.098
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd14"),
        "id" : "4868362945",
        "title" : "Digital Electronics",
```

```
    "dept" : "EE",
    "price" : 832.765,
    "subject" : "DSP"
}
```


3. Cursor Operations:

1) Limit


```
> db.Book.find().pretty().limit(2)
{
    "_id" : ObjectId("64ecdfb6848ae0682d696f51"),
    "id" : "3586824624",
    "title" : "Database Management System",
    "author" : "S.Sudharshan",
    "dept" : "IT",
    "subject" : "ADBMS",
    "price" : 806.76
}
{
    "_id" : ObjectId("64ece032848ae0682d696f52"),
    "id" : "8753698428",
    "title" : "Designing The User Interface",
    "author" : "Alan Dix",
    "dept" : "IT",
    "subject" : "HCI",
    "price" : 876.75
}
```


2) Skip


```
> db.Book.find().pretty().skip(2)
{
    "_id" : ObjectId("64eceb95caf728f8ddcefd13"),
    "id" : "23567897656",
    "title" : "Machine Design",
    "author" : "Alexander Fleming",
    "subject" : "Machine Design",
    "price" : 823.098
}
```

```
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd14"),
        "id" : "4868362945",
        "title" : "Digital Electronics",
        "dept" : "EE",
        "price" : 832.765,
        "subject" : "DSP"
}
```

3) Sort

```
> db.Book.find().sort({price:-1}).pretty()
{
        "_id" : ObjectId("64ece032848ae0682d696f52"),
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 876.75
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd14"),
        "id" : "4868362945",
        "title" : "Digital Electronics",
        "dept" : "EE",
        "price" : 832.765,
        "subject" : "DSP"
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd13"),
        "id" : "23567897656",
        "title" : "Machine Design",
        "author" : "Alexander Fleming",
        "subject" : "Machine Design",
        "price" : 823.098
}
{
        "_id" : ObjectId("64ecdfb6848ae0682d696f51"),
        "id" : "3586824624",
        "title" : "Database Management System",
        "author" : "S.Sudharshan",
```

```
        "dept" : "IT",
        "subject" : "ADBMS",
        "price" : 806.76
}

> db.Book.find().sort({price:1}).pretty()
{
        "_id" : ObjectId("64ecdfb6848ae0682d696f51"),
        "id" : "3586824624",
        "title" : "Database Management System",
        "author" : "S.Sudharshan",
        "dept" : "IT",
        "subject" : "ADBMS",
        "price" : 806.76
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd13"),
        "id" : "23567897656",
        "title" : "Machine Design",
        "author" : "Alexander Fleming",
        "subject" : "Machine Design",
        "price" : 823.098
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd14"),
        "id" : "4868362945",
        "title" : "Digital Electronics",
        "dept" : "EE",
        "price" : 832.765,
        "subject" : "DSP"
}
{
        "_id" : ObjectId("64ece032848ae0682d696f52"),
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
```

```
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 876.75
}
```

4) Update (upsert & multitrue)

```
> db.Book.update({"dept":"IT"},{$set:{"price":750}},{multi:true})
WriteResult({ "nMatched" : 2, "nUpserted" : 0, "nModified" : 2 })
> db.Book.find().pretty()

{
        "_id" : ObjectId("64ecdfb6848ae0682d696f51"),
        "id" : "3586824624",
        "title" : "Database Management System",
        "author" : "S.Sudharshan",
        "dept" : "IT",
        "subject" : "ADBMS",
        "price" : 750
}
{
        "_id" : ObjectId("64ece032848ae0682d696f52"),
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 750
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd13"),
        "id" : "23567897656",
        "title" : "Machine Design",
        "author" : "Alexander Fleming",
        "subject" : "Machine Design",
        "price" : 823.098
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd14"),
        "id" : "4868362945",
        "title" : "Digital Electronics",
```

```
        "dept" : "EE",
        "price" : 832.765,
        "subject" : "DSP"
}
>
> db.Book.update({"price":832.765},{$set:{"dept":"E&TC"}},{upsert:true})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
> db.Book.find().pretty()

{
        "_id" : ObjectId("64ecdfb6848ae0682d696f51"),
        "id" : "3586824624",
        "title" : "Database Management System",
        "author" : "S.Sudharshan",
        "dept" : "CS",
        "subject" : "ADBMS",
        "price" : 750
}
{
        "_id" : ObjectId("64ece032848ae0682d696f52"),
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 750
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd13"),
        "id" : "23567897656",
        "title" : "Machine Design",
        "author" : "Alexander Fleming",
        "subject" : "Machine Design",
        "price" : 823.098
}
{
        "_id" : ObjectId("64eceb95caf728f8ddcefd14"),
        "id" : "4868362945",
```

```
    "title" : "Digital Electronics",
    "dept" : "E&TC",
    "price" : 832.765,
    "subject" : "DSP"
}
```

5) Save

CONCLUSION:

Understand to implement data from mongodb database with the help of statement and operators.

Assignment: 2

AIM: Implement Map reduces operation with suitable example on above MongoDB database

- Aggregation framework
- Create and drop different types of indexes and explain () to show the advantage of the indexes.

PROBLEM STATEMENT /DEFINITION

Implement Map reduces operation with suitable example on above MongoDB database

- Aggregation framework
- Create and drop different types of indexes and explain () to show the advantage of the indexes.

OBJECTIVE:

To understand the concept of Mapreduce in mongodb.
To understand the concept of Aggregation in mongodb.

To implement the concept of document oriented databases.

THEORY:

- Implements the MapReduce model for processing large data sets.
- Can choose from one of several output options (inline, new collection, merge, replace, reduce)
- MapReduce functions are written in JavaScript.
- Supports non-sharded and sharded input collections.
- Can be used for incremental aggregation over large collections.
- MongoDB 2.2 implements much better support for sharded map reduce output.
- New feature in the Mongodb2.2.0 production release (August, 2012).

- Designed with specific goals of **improving performance** and **usability.**
- Returns result set inline.
- Supports **non-sharded and sharded**input collections.

- Uses a **"pipeline"** approach where objects are transformed as they pass through a series of pipeline operators such as matching, projecting, sorting, and grouping.
- **Pipeline operators need not produce one output document for every input document:** operators may also generate new documents or filter out documents.
- Map/Reduce involves two steps:
- first, map the data from the collection specified;
- second, reduce the results.

>db.createCollection("Order")

- { "ok" : 1 }

>db.order.insert({cust_id:"A123",amount:500,status:"A"})

- WriteResult({ "nInserted" : 1 })

>db.order.insert({cust_id:"A123",amount:250,status:"A"})

- WriteResult({ "nInserted" : 1 })

>db.order.insert({cust_id:"B212",amount:200,status:"A"})

- WriteResult({ "nInserted" : 1 })

>db.order.insert({cust_id:"A123",amount:300,status:"d"})

- WriteResult({ "nInserted" : 1 })

- **Map Function**
- var mapFunction1 = function()
- { emit(this.cust_id, this.amount);};
- **Reduce Function**
- var reduceFunction1 = function(key, values)
- {return Array.sum(values); };

db.order.mapReduce

(mapFunction1, reduceFunction1, {query: {status: "A" },

out: "order_totals"});

```
"result" : "order_totals",
"timeMillis" : 28, "counts" : {
        "input" :3,
        "emit" :3,
        "reduce" : 1,
        "output" : 2
},
"ok" : 1,}
```

>db.order.mapReduce(

Map Function -> function() { emit(this.cust_id, this.amount);},

Reduce Function -> function(key, values) { return Array.sum (values)}, Query à {query: { status:"A"},

Output collection à out: "order_ totals"})

```
{
    "result" : "order_totals",
    "timeMillis" : 27, "counts" : {
        "input" : 3,
        "emit" : 3,
        "reduce" : 1,
        "output" : 2
    },
    "ok" : 1,
}
```

To display result of mapReduce function use collection created in OUT.

Db.<collection name>.find();

db.order_totals.find();

{ "_id" : "A123", "value" : 750 }

{ "_id" : "B212", "value" : 200 }

Implementation of Aggregation:-

> use Teacher
switched to db Teacher
>db.Teacher.find()
{ "_id" : 101, "Name" : "Dev", "Address" : [{ "City" : "Pune", "Pin" : 444043 }], "Department" : [{ "Dept_id" : 111, "Dept_name" : "IT" }], "Salary" : 78000 }
{ "_id" : 135, "Name" : "Jennifer", "Address" : [{ "City" : "Mumbai", "Pin" : 444111 }], "Department" : [{ "Dept_id" : 112, "Dept_name" : "COMP" }], "Salary" : 65000 }
{ "_id" : 126, "Name" : "Gaurav", "Address" : [{ "City" : "Nashik", "Pin" : 444198 }], "Department" : [{ "Dept_id" : 112, "Dept_name" : "COMP" }], "Salary" : 90000 }
{ "_id" : 175, "Name" : "Shree", "Address" : [{ "City" : "Nagpur", "Pin" : 444158 }], "Department" : [{ "Dept_id" : 113, "Dept_name" : "ENTC" }], "Salary" : 42000 }
{ "_id" : 587, "Name" : "Raman", "Address" : [{ "City" : "Banglore", "Pin" : 445754 }], "Department" : [{ "Dept_id" : 113, "Dept_name" : "ENTC" }], "Salary" : 79000 }
{ "_id" : 674, "Name" : "Mandar", "Address" : [{ "City" : "Jalgaon", "Pin" : 465487 }], "Department" : [{ "Dept_id" : 111, "Dept_name" : "IT" }], "Salary" : 88000 }
{ "_id" : 573, "Name" : "Manish", "Address" : [{ "City" : "Washim", "Pin" : 547353 }], "Department" : [{ "Dept_id" : 112, "Dept_name" : "COMP" }], "Salary" : 65000 }

>db.Teacher.aggregate([
... {$group:{_id:"$Department",totalsalary:{$sum:"$Salary"}}}...])
{
"result" : [
{
"_id" : [
{
"Dept_id" : 113,
"Dept_name" : "ENTC"
}

```
                ,
                "totalsalary" : 121000
        },
```

```
{
                "_id" : [
                        {
                                "Dept_id" : 112,
                                "Dept_name" : "COMP"],
                "totalsalary" : 220000
        },
        {
                "_id" : [
                        {
                                "Dept_id" : 111,
                                "Dept_name" : "IT"
                        }
                ],
                "totalsalary" : 166000
        }
    ],
    "ok" : 1
}
>db.Teacher.aggregate([
{$group:{_id:"$Department",totalsalary:{$sum:"$Salary"}}},{$group:{_id:"$_id.Departm
ent",AvgSal:{$sum:"$totalsalary"}}}])
{ "result" : [ { "_id" : [ ], "AvgSal" : 507000 } ], "ok" : 1 }
>db.Teacher.aggregate([
{$group:{_id:"$Department",totalsalary:{$sum:"$Salary"}}},{$match:{totalsalary:{$gte:2
00000}}}])
{
    "result" : [
        {
                "_id" : [
                        {
                                "Dept_id" : 112,
                                "Dept_name" : "COMP"
                        }
                ],
                "totalsalary" : 220000
        }
    ],
    "ok" : 1
}
```

>db.Teacher.aggregate([{$group:{_id:"$Department",totalsalary:{$sum:"$Salary"}}}, { $sort:{totalsalary:1}}])

```
{
        "result" : [
                {
"_id" : [
                                                                                {
                                                                                }
```

>db.Teacher.aggregate([{$group:{_id:"$Department",totalsalary:{$sum:"$Salary"}}},{$group:{_id:"$_id.Department",AvgSal:{$sum:"$totalsalary"}}}])

```
{ "result" : [ { "_id" : [ ], "AvgSal" : 507000 } ], "ok" : 1 }
```

>db.Teacher.aggregate([{$group:{_id:"$Department",totalsalary:{$sum:"$Salary"}}},{$match:{totalsalary:{$gte:200000}}}])

```
{
        "result" : [
                {
                        "_id" : [
                                {
                                        "Dept_id" : 112,
                                        "Dept_name" : "COMP"
                                }
                        ]                       2
                        ,                       0
                        "                       0
                        t                       0
                        o               "Dept_id" : 113,
                        t               } "Dept_name" : "ENTC"
                        a       ],
                        l       "ok
                        s       " :
                        a       1
                } l
                        a
                        r
                        y
                        "

                        :

                        2
```

```
                {
                        "_id" : [
                                {
                                        "Dept_id" : 111,
                                        "Dept_name" : "IT"
                                }
                        ],
                        "totalsalary" : 166000
                },
                {
                        "_id" : [
                                {
                                        "Dept_id" : 112,
                                        "Dept_name" : "COMP"
                                }
                        ],
                        "totalsalary" : 220000
                }
        ],
        "ok" : 1
}
```


>db.Teacher.aggregate([{$group:{_id:"$Department",totalsalary:{$sum:"$Salary"}}}, { $group: { _id:"$_id.Department", big: { $last: "$_id.Dept_name" }, bigsalary: { $last:"$totalsalary"}, small: { $first:"$_id.Dept_name"}, smallsalary: { $first:"$totalsalary"} }}])


```
{
        "result" : [
                {
                        "_id" : [ ],
                        "big" : [
                                "IT"
                        ],
                        "bigsalary" : 166000,
                        "small" : [
                                "ENTC"
                        ],
                        "smallsalary" : 121000
                }
        ],
        "ok" : 1
}
```

REFERENCE BOOK:

Kristina Chodorow, MongoDB The definitive guide, O‟Reilly Publications, ISBN:978-93-5110-269-4,2nd Edition.

Title: Implement Map-reduce and aggregation, indexing with suitable example in MongoDB. Demonstrate the following:
• Aggregation framework
• Create and drop different types of indexes and explain () to show the advantage of the indexes.
****

1) Switch database

```
> use assign1
switched to db assign1
```

2) Create Collection

```
> db.createCollection("Book");
{ "ok" : 1 }
```

3) Insert documents into collection

```
> db.Book.insert({id:"3586824624",title:"Database Management
System",author:"S.Sudharshan",dept:"IT",subject:"ADBMS",price:756.76})
WriteResult({ "nInserted" : 1 })
> db.Book.insert({id:"8753698428",title:"Designing The User Interface",author:"Alan
Dix",dept:"IT",subject:"HCI",price:876.75})
WriteResult({ "nInserted" : 1 })
> db.Book.insert([{id:"2376574365",title:"Machine Design",author:"Saul
        Fenster",dept:"ME",subject:"Machine
        Design",price:975},{id:"4868362945",title:"Digital
        Electronics",dept:"EE",price:832.765},{id:298374574,title:"Engineering
        Thermodyanamics",author:"P.K.Nag",dept:"ME",price:732.98}])
```

4) Retrieve and Display Documents

```
> db.Book.find().pretty()
        {
         "_id" : ObjectId("64ece032848ae0682d696f52"),
        "id" : "8753698428",
        "title" : "Designing The User Interface",
        "author" : "Alan Dix",
        "dept" : "IT",
        "subject" : "HCI",
        "price" : 876.75
        }
        {
        "_id" : ObjectId("64eceb95caf728f8ddcefd13"),
        "id" : "2376574365",
        "title" : "Machine Design",
        "author" : "Saul Fenster",
        "dept" : "ME",
        "subject" : "Machine Design",
        "price" : 975
        }
        {
        "_id" : ObjectId("64eceb95caf728f8ddcefd14"),
        "id" : "4868362945",
        "title" : "Digital Electronics",
        "dept" : "EE",
        "price" : 832.765
        }
        {
        "_id" : ObjectId("64eceb95caf728f8ddcefd15"),
        "id" : 298374574,
        "title" : "Engineering Thermodyanamics",
        "author" : "P.K.Nag",
        "dept" : "ME",
        "price" : 732.98
        }
```

**************************Map Reduce Function***********************

```
> var mf=function(){emit(this.dept,this.price)}
> var rf=function(Department,Total){return Array.sum(Total)}
> db.Book.mapReduce(mf,rf,{query:{dept:"IT"},out:"Total"})
{ "result" : "Total", "ok" : 1 }
> db.Total.find()
{ "_id" : "IT", "value" : 4165.76 }
var mf=function(){emit(this.dept,this.price)}
> var rf=function(Department,Total){return Array.avg(Total)}
> db.Book.mapReduce(mf,rf,{query:{dept:"IT"},out:"Average"})
{ "result" : "Average", "ok" : 1 }
> db.Average.find()
{ "_id" : "IT", "value" : 833.152 }
```

**************************Aggregate Operation (SUM)***********************

```
>
db.Book.aggregate([{$match:{"dept":"IT"}},{$group:{_id:{"department":"$dept"},total:{$sum:"$pri
ce"}}}])
{ "_id" : { "department" : "IT" }, "total" : 750 }
```

**************************Aggregate Operation (AVG)***********************

```
> db.Book.aggregate([{$match:{"author":"Alan
Dix"}},{$group:{_id:{"department":"$dept"},total:{$avg:"$price"}}}])
{ "_id" : { "department" : "IT" }, "total" : 750 }
```

**************************Aggregate Operation (MAX/MIN)***********************

```
>
db.Book.aggregate([{$match:{"dept":"IT"}},{$group:{_id:{"department":"$dept"},min:{$min:"$pric
e"}}}])
{ "_id" : { "department" : "IT" }, "min" : 701 }
>
>
db.Book.aggregate([{$match:{"dept":"IT"}},{$group:{_id:{"department":"$dept"},max:{$max:"$pri
ce"}}}])
{ "_id" : { "department" : "IT" }, "max" : 986 }
>
```

**********************Creating Index ************

```
> db.Book.createIndex({"price":1})
{
        "numIndexesBefore" : 1,
        "numIndexesAfter" : 2,
        "createdCollectionAutomatically" : false,
        "ok" : 1
}
```

***************************Display Index*******************

```
> db.Book.getIndexes()
[
        {
                "v" : 2,
                "key" : {
                        "_id" : 1
                },
                "name" : "_id_"
        },
        {
                "v" : 2,
                "key" : {
                        "price" : 1
                },
                "name" : "price_1"
        }
]
>
```

***************************Drop Index***************************

```
> db.Book.dropIndexes({price:1})
{ "nIndexesWas" : 2, "ok" : 1 }
> db.Book.getIndexes()
[ { "v" : 2, "key" : { "_id" : 1 }, "name" : "_id_" } ]
>
```

CONCLUSION:

Understand to mapreduce operation in mongodb

Assignment 3

Aim : Case Study: Design conceptual model using Star and Snowflake schema for any one database **Problem statement**: Design conceptual model using Star and Snowflake schema for any one database **OBJECTIVE:**

1. To understand concepts of multidimensional data.
2. To understand the the relational implementation of the multidimensional data model is typically a star schema, or a snowflake schema.

Theory:

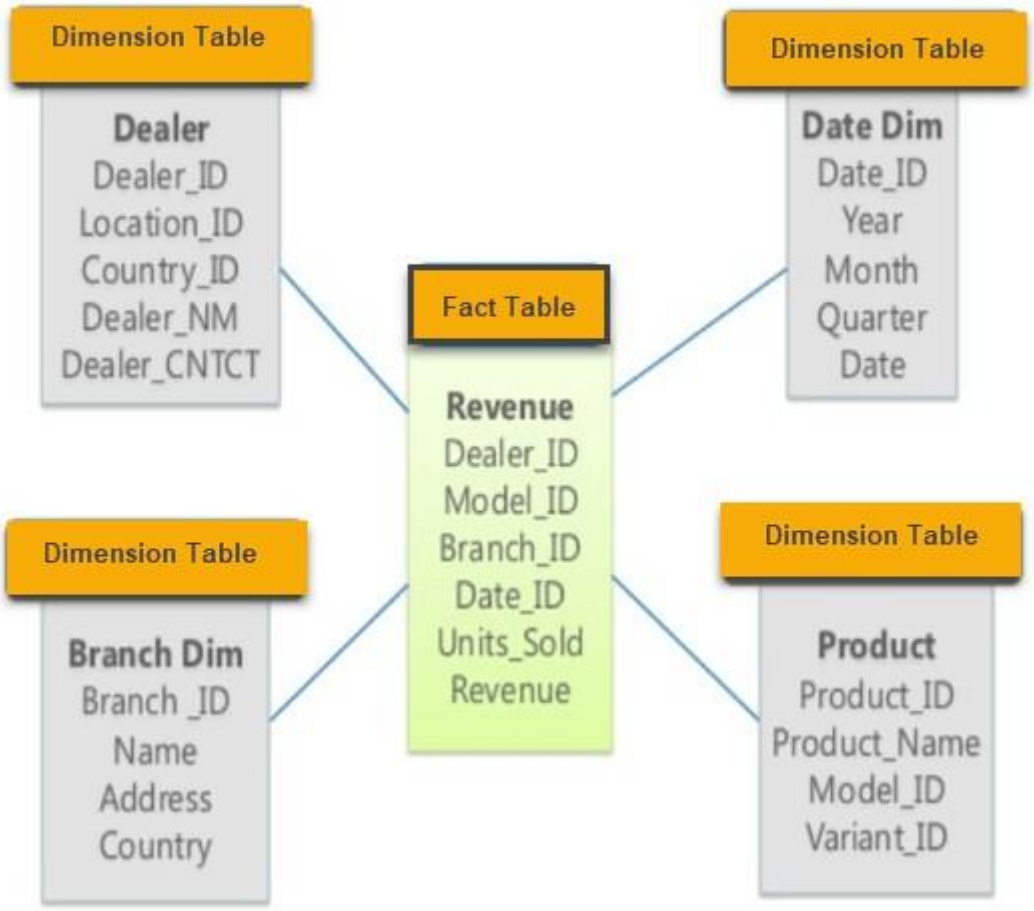
The data warehouses are considered modern ancient techniques, since the early days for the relational databases, the idea of the keeping a historical data for reference when it needed has been originated, and the idea was primitive to create archives for the historical data to save these data, despite of the usage of a special techniques for the recovery of these data from the different storage modes. This research applied of structured databases for a trading company operating across the continents, has a set of branches each one has its own stores and showrooms, and the company branch‟s group of sections with specific activities, such as stores management, showrooms management, accounting management, contracts and other departments. It also assumes that the company center exported software to manage databases for all branches to ensure the safety performance, standardization of processors and prevent the possible errors and bottlenecks problems. Also the research provides this methods the best requirements have been used for the applied of the data warehouse (DW), the information that managed by such an applied must be with high accuracy. It must be emphasized to ensure compatibility information and hedge its security, in schemes domain, been applied to a comparison between the two schemes (Star and Snowflake Schemas) with the concepts of multidimensional database. It turns out that Star Schema is better than Snowflake Schema in (Query complexity, Query performance, Foreign Key Joins),And finally it has been concluded that Star Schema center fact

and change, while Snowflake Schema center fact and not change

Example:

1. *Star Schema*

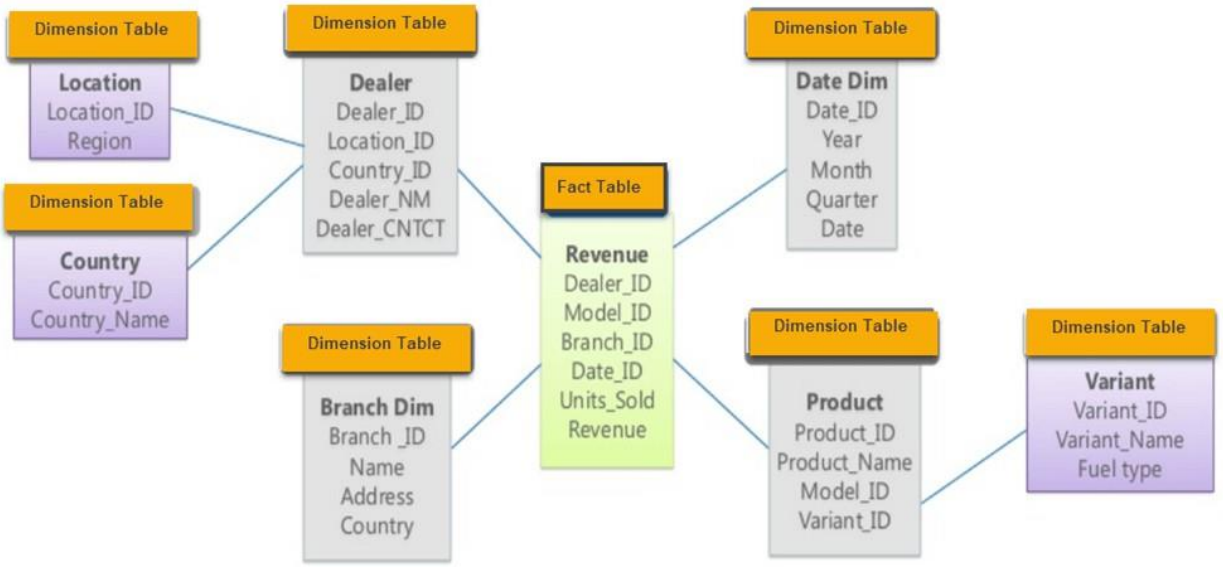
Star Schema in data warehouse, in which the center of the star can have one fact table and a number of associated dimension tables. It is known as star schema as its structure resembles a star. The Star Schema data model is the simplest type of Data Warehouse schema. It is also known as Star Join Schema and is optimized for querying large data sets



2. **Snowflake Schema?**

Snowflake Schema in data warehouse is a logical arrangement of tables in a multidimensional database such that the ER diagram resembles a snowflake shape. A Snowflake Schema is an extension of a Star Schema, and it adds additional dimensions. The dimension tables are normalized which splits data into additional tables.

In the following Snowflake Schema example, Country is further normalized into an individual table.



- Multidimensional schema is especially designed to model data warehouse systems
- The star schema is the simplest type of Data Warehouse schema. It is known as star schema as its structure resembles a star.
- Comparing Snowflake vs Star schema, a Snowflake Schema is an extension of a Star Schema, and it adds additional dimensions. It is called snowflake because its diagram resembles a Snowflake.
- In a star schema, only single join defines the relationship between the fact table and any dimension tables.
- Star schema contains a fact table surrounded by dimension tables.
- Snowflake schema is surrounded by dimension table which are in turn surrounded by dimension table
- A snowflake schema requires many joins to fetch the data.
- Comparing Star vs Snowflake schema, Start schema has simple DB design, while Snowflake schema has very complex DB design.

REFERENCE BOOK:

Jiawei Han, Micheline Kamber, Jian Pei “Data Mining: concepts and techniques”, 2nd Edition,
Publisher: Elsevier/Morgan Kaufmann.

CONCLUSION:

Understand to concept of multidimensional data.

PART C:

MINI PROJECT

Mini Project

AIM: Build the mini project based on the relevant applicable concepts of Machine Learning / DAA / ADBMS by forming teams of around 3 to 4 students.

A] Sample ML mini–project Format:

PROBLEM STATEMENT /DEFINITION

Design and Implement any Data science Application using Python. Obtain Data, Scrub data (Data cleaning), Explore data, Prepare and validate data model and Interpret data (Data Visualization). Visualize data using any visualization tool like Matplotlib, ggplot, Tableau etc. Prepare Project Report.

OBJECTIVE:

1. To explore the Data science project life cycle.
2. To identify need of project and define problem statement.
3. To extract and process data.
4. To interpret and analyze results using data visualization.

Mini Project Report Format:

Abstract

Acknowledgement

List of Tables & Figures

Contents

1. Introduction
   - 1.1 Purpose, Problem statement
   - 1.2 Scope, Objective
   - 1.3 Definition, Acronym, and Abbreviations
   - 1.4 References
2. Literature Survey

2.1 Introduction

2.2 Detail Literature survey

2.4 Findings of Literature survey

3. System Architecture and Design

3.1 Detail Architecture

3.2 Dataset Description

3.3 Detail Phases

3.4Algorithms

4. Experimentation and Results

4.1 Phase-wise results

4.2 Explanation with example

4.3 Comparison of result with standard

4.4Accuracy

4.5Visualization

4.6 Tools used

5. Conclusion and Future scope

5.1 Conclusion

5.2 Future scope

References

Annexure:

A. GUIs / Screen Snapshot of the System Developed
B. Implementation /code

B]**Sample ADBMS mini–project Format:**

PROBLEM STATEMENT /DEFINITION

Build the mini project based on the requirement document and design prepared as a part of Database Management Lab in second year.
Form teams of around 3 to 4 people.
A. Develop the application: Build a suitable GUI by using forms and placing the controls on it for any application. Proper data entry validations are expected.
B. Add the database connection with front end. Implement the basic CRUD operations.
C. Prepare and submit report to include Title of the Project, Abstract, List the hardware and software requirements at the backend and at the front end, Source Code, Graphical User Interface, Conclusion.

OBJECTIVE:

3. To understand applications of document-oriented database by implementing mini project.
4. To learn effective UI designs.
5. To learn to design & implement database system for specific domain.
6. To learn to design system architectural & flow diagram.

.

Mini Project Report Format:

Abstract

Acknowledgement

List of Tables & Figures

Contents

1. Introduction
   - 1.1 Purpose
   - 1.2 Scope
   - 1.3 Definition, Acronym, and Abbreviations
   - 1.4 References
   - 1.5 Developers‟ Responsibilities: An Overview
2. General Description
   - 2.1 Product Function Perspective
   - 2.2 User Characteristics.
   - 2.4 General Constraints
   - 2.5 Assumptions and Dependencies
3. Specific Requirements
   - 3.1 Inputs and Outputs
   - 3.2 Functional Requirements
   - 3.3 Functional Interface Requirements
   - 3.3 Performance Constraints
   - 3.4 Design Constraints
   - 3.6 Acceptance criteria

4. System Design

5. System Implementation

   5.1 Hardware and Software Platform description

   5.2 Tools used

   5.3 System Verification and Testing (Test Case Execution)

   5.4 Future work / Extension

   5.5 Conclusion

References

Annexure:

A. GUIs / Screen Snapshot of the System Developed